

基于几何判定与分层优化的烟幕弹协同投放策略

摘要

针对多架无人机利用烟幕干扰弹延迟来袭导弹识别真目标的问题,本文以无人机航向、飞行速度、投放时刻、起爆延迟和任务分配为决策变量,以各导弹对应的累计有效遮蔽时间之和为目标,建立运动学、完整目标遮蔽判定与分层优化相结合的协同投放模型。模型显式描述导弹、无人机、烟幕弹及起爆后云团的运动过程,并离散圆柱形真目标的迎弹侧面与顶面;当全部可见采样点到至少一个有效烟幕球的视线距离满足阈值时,判定该时刻形成完整遮蔽。在此基础上,分别求解五个递进问题。

对于**问题一**,依据给定航向、速度、投放时刻和起爆延迟进行解析轨迹计算,并在加密时间网络上积分遮蔽指示函数。得到投放点为 (17620,0,1800),起爆点为 (17188,0,1736.496),累计有效遮蔽时间为 1.390 s,为后续参数优化提供基准。

对于**问题二**,将 FY1 的航向、速度、投放时刻和起爆延迟构成四维连续决策向量。采用 Logistic 接近度构造连续代理目标,利用差分进化完成全局搜索,再以完整圆柱离散模型进行局部精修和严格验收。最优累计遮蔽时间为 4.580 s,较给定策略提高约 229.5%。

对于**问题三**,在共享航向和速度的条件下,以非负时间增量表示三枚烟幕弹的投放时刻,从变量结构上满足相邻投放间隔不少于 1 s 的约束;目标函数按三个云团遮蔽区间的并集计时,避免重复累计。优化后联合遮蔽时间为 6.410 s。

对于**问题四**,先分别生成 FY1、FY2、FY3 的单弹可行候选,再围绕联合遮蔽区间进行协同精修,使不同来向形成的云团在时间上接续、空间上互补。三机各投放一枚烟幕弹时,对 M1 的累计有效遮蔽时间达到 11.170 s。

对于**问题五**,建立五架无人机、三枚来袭导弹和多枚烟幕弹之间的混合任务分配模型。采用“连续候选生成—组合筛选—分块精修—边际贡献删弹—缺口驱动增弹”的分层算法,并用 0-1 整数规划在离散候选库上认证组合最优性。最终保留 11 枚具有正边际贡献的烟幕弹,M1、M2、M3 的累计遮蔽时间分别为 9.960 s、12.500 s 和 6.120 s,总目标值为 28.580 s;分支定界最优性间隙为 0,表明该方案是既定候选集和离散精度下的全局最优组合。

本文的创新点主要有三方面:一是以圆柱目标全部可见表面点为对象建立严格遮蔽判据,避免仅用目标中心判断造成的高估;二是利用连续代理目标引导非光滑搜索,并以严格几何模型复核,实现搜索效率与判定精度的分离;三是将多机多弹连续轨迹优化与离散任务分配分层处理,同时通过边际贡献删弹和整数规划给出可解释的资源配置结果。加密时间网格与圆柱采样复算表明,五问粗、细网格结果差异均不超过 0.060 s。100 次执行误差模拟所得总时长均值为 19.895 s,说明名义最优解对航向与时序扰动较敏感,实际部署时应进一步设置安全裕度或引入鲁棒优化。

关键词: 烟幕干扰弹;完整目标几何判定;差分进化;协同任务分配;0-1 整数规划

一、 问题重述

某型无人机可在接到任务后瞬时调整航向，之后以 $70 \sim 140 \text{ m/s}$ 的速度等高度匀速直线飞行。无人机投放的烟幕干扰弹脱离载机后做抛体运动，起爆后瞬时形成半径为 10 m 的球形有效烟幕区域。云团中心以 3 m/s 的速度竖直下沉，起爆后 20 s 内可提供有效遮蔽。同一架无人机连续投放两枚烟幕弹的时间间隔不得小于 1 s 。

三枚来袭导弹均以 300 m/s 匀速飞向位于坐标原点的假目标，真目标为下底面圆心位于 $(0, 200, 0)$ 、半径 7 m 、高 10 m 的圆柱。需要针对五种情形确定无人机航向、速度、投放时刻和起爆延迟，使导弹观察真目标的视线被烟幕遮蔽的累计时间尽可能长。

五个问题由单弹给定策略计算逐步扩展为单机单弹优化、单机三弹优化、三机单弹协同以及五机多弹多目标协同。核心难点在于：遮蔽函数具有明显的 $0-1$ 跳变；同一烟幕可能参与多个时段或多个目标点的联合遮蔽；问题 5 同时含有连续航迹变量、连续投弹变量和离散任务分配变量。

二、 问题分析

2.1 统一运动关系

导弹、无人机、烟幕弹和烟幕云团均具有明确的运动规律。无人机航向和速度确定后，投放点可直接计算；烟幕弹在水平方向继承无人机速度，在竖直方向做自由落体；起爆后不再保留原水平速度，而形成定点生成并竖直下沉的烟幕云团。因此，各阶段位置均可表示为时间和决策变量的显式函数。

2.2 遮蔽判定的几何本质

导弹是否发现真目标，取决于导弹与真目标可见表面之间的视线是否穿过有效烟幕球。仅判断目标中心会忽略圆柱尺寸，故本文离散圆柱的可见侧面和顶面，对每个可见采样点分别计算烟幕中心到导弹-采样点线段的最短距离。当所有可见点均被至少一个烟幕球覆盖时，认定该时刻真目标被完整遮蔽。

2.3 非光滑优化与分层求解

严格遮蔽时间在大部分参数区域恒为零，直接使用智能算法容易陷入平台。本文以中心视线附近的连续 Logistic 接近度作为搜索代理，负责把候选带入有效区域；最终目标值始终由严格完整目标判据计算。连续参数搜索采用差分进化方法 [1]。对于问题 5，先生成无人机-导弹候选任务，再进行组合选择和连续精修，随后删除冗余弹，并针对遮蔽空缺增量增加烟幕弹。最后用整数规划对有限候选库进行组合最优性认证。

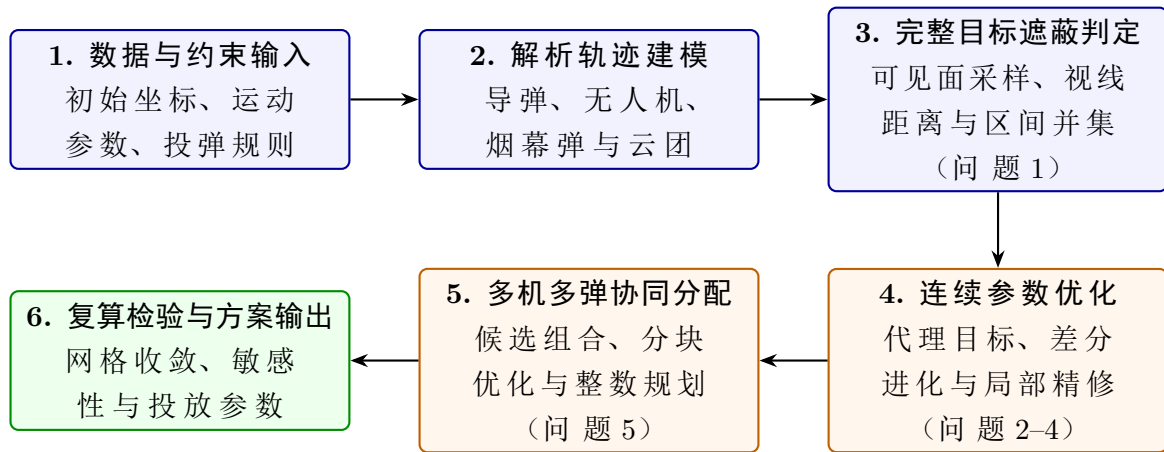


图 1 烟幕干扰弹协同投放的总体建模与分层求解流程

2.4 五个问题的递进关系与求解重点

五个问题并非彼此独立，而是沿着“判定模型验证—连续参数优化—同平台时序耦合—多平台空间协同—多目标资源配置”的路线逐层增加复杂度。

问题 1：给定策略计算。全部运动参数均已给定，主要任务是将解析轨迹代入遮蔽判据并积分。该问用于校验坐标方向、投放点、起爆点和完整目标判定，为后续优化提供基准算例。

问题 2：单机单弹连续优化。在问题 1 的模型上放开航向、速度、投放时刻和引信延迟，形成四维非光滑优化问题。求解重点是利用连续代理跨越严格目标的大面积零值平台，并对边界解进行物理解释。

问题 3：单机三弹时序优化。三枚弹共享航向和速度，并受相邻投放间隔约束。目标不能采用单弹时长之和，而应协调各云团的有效时间窗并计算遮蔽区间并集，以减少重复遮蔽。

问题 4：三机空间协同。三架无人机可以从不同方向接近视线，各自投放一枚烟幕弹。该问需先构造单机候选，再在联合遮蔽目标下调整全部平台，识别不同云团之间的拼接覆盖和时间接力。

问题 5：多目标混合优化。五架无人机、三枚导弹和每机至多三枚烟幕弹共同引入连续航迹、连续时序、任务分配和投弹数量变量。为控制搜索规模，采用连续优化生成候选、组合优化选择候选、局部搜索修正参数的分层结构。

从评价口径看，问题 1-4 均针对指定导弹计算完整目标的累计遮蔽时间；问题 5 则最大化三枚导弹累计遮蔽时间之和。由于题目允许遮蔽区间不连续，目标函数对各有效区间直接求并集长度，而不以最长连续区间代替累计时间。该统一口径保证五问结果可以相互校验，也避免将单枚烟幕弹的独立遮蔽时间误加为联合遮蔽时间。

三、 模型假设

在不改变题目主要物理关系的前提下，作如下假设：

- 1) 导弹在研究时段内保持 300 m/s 匀速直线飞行，方向始终指向假目标原点；
- 2) 无人机接到任务后可瞬时改变航向，随后保持定高、定速、直线飞行，不计转向过渡过程；
- 3) 烟幕弹脱离无人机后水平方向速度不变，竖直方向仅受重力作用，取 $g = 9.8 \text{ m/s}^2$ ；
- 4) 烟幕弹起爆后瞬时形成球形有效区域，云团不考虑水平漂移，仅以 3 m/s 匀速下沉；
- 5) 云团有效半径为 10 m，有效期为起爆后 20 s；为避免贴地起爆的浮点误判，数值计算要求起爆高度不低于 1 m，但最终方案最低高度远高于该阈值；
- 6) 不考虑无人机、导弹和烟幕弹之间的碰撞，也不考虑风场、空气阻力和烟幕扩散形变；
- 7) 不同烟幕弹的有效区域可以重叠，完整遮蔽由所有有效云团的覆盖并集共同决定。

四、符号说明

表 1 主要符号及含义

符号	含义	单位
$\mathbf{m}_j^0$	导弹 M_j 的初始位置	m
$\mathbf{u}_k^0$	无人机 FY k 的初始位置	m
v_M	导弹速度，取 300	m/s
v_k, θ_k	无人机 FY k 的速度和航向角	m/s, °
t_{ki}^d	第 i 枚烟幕弹的投放时刻	s
τ_{ki}	投放至起爆的延迟	s
t_{ki}^b	起爆时刻， $t_{ki}^b = t_{ki}^d + \tau_{ki}$	s
$\mathbf{d}_{ki}, \mathbf{b}_{ki}$	投放点与起爆点	m
$\mathbf{c}_{ki}(t)$	起爆后烟幕云团中心位置	m
R_s	烟幕有效半径，取 10	m
$H_j(t)$	导弹 M_j 在时刻 t 的完整遮蔽指标	—
T_j	对导弹 M_j 的累计有效遮蔽时间	s

五、运动学与遮蔽判定模型

5.1 导弹运动模型

设假目标为原点，导弹 M_j 初始位置为 $\mathbf{m}_j^0$ 。其单位飞行方向为

$$\mathbf{e}_j^M = -\frac{\mathbf{m}_j^0}{\|\mathbf{m}_j^0\|_2}. \quad (1)$$

因此导弹位置为

$$\mathbf{m}_j(t) = \mathbf{m}_j^0 + v_M t \mathbf{e}_j^M, \quad 0 \leq t \leq T_j^M = \frac{\|\mathbf{m}_j^0\|_2}{v_M}. \quad (2)$$

三枚导弹到达假目标的时间约为 66.999 s、63.750 s 和 60.367 s。

5.2 无人机与烟幕弹运动模型

无人机 FY k 航向角为 θ_k ，速度为 v_k ，水平单位方向为

$$\mathbf{e}_k^U = (\cos \theta_k, \sin \theta_k, 0). \quad (3)$$

无人机位置为

$$\mathbf{u}_k(t) = \mathbf{u}_k^0 + v_k t \mathbf{e}_k^U. \quad (4)$$

第 i 枚烟幕弹在 t_{ki}^d 投放，其投放点为

$$\mathbf{d}_{ki} = \mathbf{u}_k^0 + v_k t_{ki}^d \mathbf{e}_k^U. \quad (5)$$

起爆时刻为 $t_{ki}^b = t_{ki}^d + \tau_{ki}$ ，起爆点为

$$\mathbf{b}_{ki} = \mathbf{u}_k^0 + v_k t_{ki}^b \mathbf{e}_k^U - \left(0, 0, \frac{1}{2} g \tau_{ki}^2\right). \quad (6)$$

起爆后云团中心为

$$\mathbf{c}_{ki}(t) = \mathbf{b}_{ki} - (0, 0, 3(t - t_{ki}^b)), \quad t_{ki}^b \leq t \leq t_{ki}^b + 20. \quad (7)$$

5.3 点到视线段距离

设某时刻导弹位置为 $\mathbf{m}$ ，目标采样点为 $\mathbf{q}$ ，云团中心为 $\mathbf{c}$ 。导弹到目标点的视线段可写为

$$\ell(\lambda) = \mathbf{m} + \lambda(\mathbf{q} - \mathbf{m}), \quad 0 \leq \lambda \leq 1. \quad (8)$$

投影系数为

$$\lambda^* = \text{clip}_{[0,1]} \frac{(\mathbf{c} - \mathbf{m})^\top (\mathbf{q} - \mathbf{m})}{\|\mathbf{q} - \mathbf{m}\|_2^2}. \quad (9)$$

云团中心到视线段的最短距离采用计算几何中的投影截断方法 [2]，即

$$d(\mathbf{c}, \overline{\mathbf{m}\mathbf{q}}) = \|\mathbf{c} - \mathbf{m} - \lambda^*(\mathbf{q} - \mathbf{m})\|_2. \quad (10)$$

当 $d \leq R_s$ 且云团处于有效期内时，该云团可以遮挡导弹对采样点 $\mathbf{q}$ 的视线。

图2给出竖直剖面内的遮蔽关系。只有当烟幕球与导弹到目标点的有限视线段相交时，该目标点才被遮挡；投影系数截断到 $[0, 1]$ 可排除位于导弹后方或目标后方的云团。

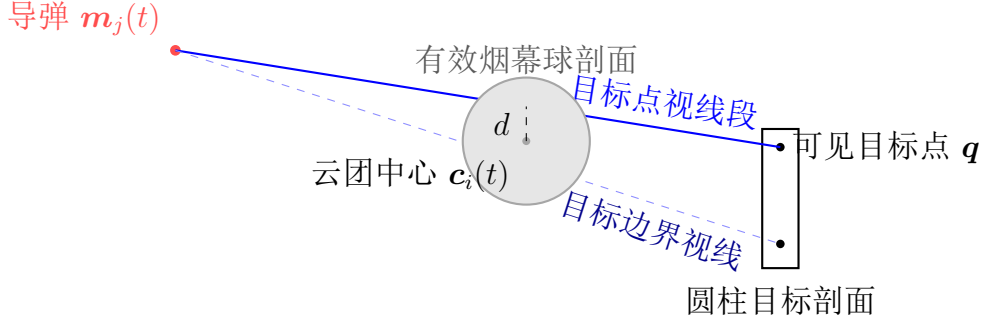


图 2 烟幕云团遮挡导弹视线的几何关系

5.4 完整圆柱目标遮蔽

将圆柱侧面按圆周角和高度离散，并在顶面设置若干同心圆采样点。根据导弹位置剔除不可见侧面点，得到时刻 t 的可见点集合 $\mathcal{Q}_j(t)$ 。定义

$$h_{ijq}(t) = \mathbf{1} \left\{ d(\mathbf{c}_i(t), \overline{\mathbf{m}_j(t)\mathbf{q}}) \leq R_s \right\}. \quad (11)$$

多云团联合遮蔽指标为

$$H_j(t) = \min_{\mathbf{q} \in \mathcal{Q}_j(t)} \max_{i \in \mathcal{A}(t)} h_{ijq}(t), \quad (12)$$

其中 $\mathcal{A}(t)$ 为时刻 t 的有效云团集合。累计遮蔽时间为

$$T_j = \int_0^{T_j^M} H_j(t) dt. \quad (13)$$

数值计算采用离散时间网格和梯形积分，最终结果再用更小时间步长与更密集目标采样复算。

5.4.1 可见表面采样与联合覆盖逻辑

真目标是半径为 7 m、高 10 m 的圆柱体。若对整个封闭表面统一采样，不可见背面的点也会进入约束，容易低估实际遮蔽时间。为此，在每个离散时刻先根据导弹位置判断圆柱侧面的迎弹半圆，再对迎弹侧面和顶面进行采样。侧面点写为

$$\mathbf{q}_{ab} = (7 \cos \varphi_a, 200 + 7 \sin \varphi_a, z_b), \quad (14)$$

其中 φ_a 仅保留法向量朝向导弹的角度, $z_b \in [0, 10]$ 。顶面采用若干同心圆采样, 补充仅采样外轮廓可能遗漏的顶部视线。正式计算取 24 个圆周方向和 5 个高度层, 高密度复算将其提高到 48 个圆周方向和 9 个高度层; 收敛检验进一步采用 72 个方向和 13 个高度层。

侧面点的外法向量为 $\mathbf{n}(\varphi_a) = (\cos \varphi_a, \sin \varphi_a, 0)$ 。记圆柱轴线上与 $\mathbf{q}_{ab}$ 等高的点为 $\mathbf{o}_b = (0, 200, z_b)$, 则侧面可见条件取为

$$\mathbf{n}(\varphi_a)^\top (\mathbf{m}_j(t) - \mathbf{o}_b) > 0. \quad (15)$$

导弹高度高于圆柱顶面时保留顶面采样点, 否则只保留迎弹侧面点。该筛选排除了被圆柱自身遮挡的背面点, 又保留了导弹在高空俯视时能够观察到的顶部区域。

对每个可见点, 不要求由同一烟幕云团独立完成遮蔽。式中先对有效云团取最大值, 再对全部可见点取最小值, 允许多个云团分别覆盖目标表面的不同区域。这一“先并后交”的逻辑与多枚烟幕弹协同的物理含义一致。若反过来先判断单个云团是否覆盖完整目标, 再对云团取并集, 会遗漏多个云团拼接形成完整遮蔽的情形。

在离散时刻 $t_n = n\Delta t$ 上, 累计时间近似为

$$\hat{T}_j = \Delta t \left[\frac{H_j(t_0) + H_j(t_N)}{2} + \sum_{n=1}^{N-1} H_j(t_n) \right]. \quad (16)$$

由于 $H_j(t)$ 为 0-1 函数, 误差主要集中在遮蔽区间端点附近。后文依据数值分析中的网格加密思想 [3], 同时给出时间步长和空间采样密度的收敛检验, 以判断三位小数的报告精度是否合理。

5.4.2 统一决策变量与约束集合

对无人机 FY k , 记航线变量为 (θ_k, v_k) , 第 i 枚烟幕弹的投放与延迟变量为 (t_{ki}^d, τ_{ki}) 。所有问题共享以下可行域:

$$0 \leq \theta_k < 2\pi, \quad 70 \leq v_k \leq 140, \quad (17)$$

$$t_{ki}^d \geq 0, \quad \tau_{ki} \geq 0, \quad (18)$$

$$t_{k,i+1}^d - t_{ki}^d \geq 1, \quad (19)$$

$$z_{ki}^b = z_k^0 - \frac{1}{2}g\tau_{ki}^2 \geq 1. \quad (20)$$

最后一个不等式避免烟幕弹在到达地面后起爆。问题 2 只含一枚烟幕弹, 投放间隔约束自然消失; 问题 4 每架无人机仅投一枚弹, 同样不存在机内时序约束。问题 5 还需满足每架无人机最多投放三枚弹。对于三枚导弹, 优化目标统一写为

$$\max F(\mathbf{x}) = \sum_{j \in \mathcal{J}} T_j(\mathbf{x}), \quad (21)$$

其中问题 1-4 的集合 $\mathcal{J}$ 只含题目指定导弹, 问题 5 取 $\mathcal{J} = \{1, 2, 3\}$ 。

5.5 连续搜索代理

严格指标在绝大多数参数空间为零。为增强搜索方向性，定义连续接近度

$$S_j(t) = \max_i \frac{1}{1 + \exp[(d_{ij}(t) - R_s)/\sigma]}, \quad \sigma = 2, \quad (22)$$

并以 $\sum_j \int S_j(t) dt$ 引导差分进化。代理指标只用于生成候选，正式结果始终由 $H_j(t)$ 评价。

差分进化的个体直接编码连续决策向量。对第 r 个个体 $\mathbf{x}_r^{(g)}$ ，随机选取互异个体 a, b, c ，形成变异向量

$$\mathbf{v}_r^{(g)} = \mathbf{x}_a^{(g)} + F(\mathbf{x}_b^{(g)} - \mathbf{x}_c^{(g)}). \quad (23)$$

随后按交叉概率 C_R 逐维生成试验向量，并至少保留一个变异分量。超出速度、时间或起爆高度边界的分量投影回可行域。若试验向量的代理得分优于父代，则进入下一代。全局搜索结束后，将代理得分靠前的个体送入严格模型验收，再围绕严格得分最优者进行坐标局部搜索。局部搜索依次对每一维作正、负方向扰动；若一轮没有改进，则将步长减半，直到达到精度阈值或最大轮数。

这一双层评价机制解决了两个矛盾：一方面，连续代理提供非零的搜索梯度，使种群能够接近导弹视线；另一方面，最终选择不依赖代理函数，避免出现“中心视线接近但圆柱边缘仍可见”的虚假高分。对问题 3–5，所有候选还要经过投弹间隔、起爆高度和最大投弹数检查，违反约束的个体直接赋予惩罚值。

5.6 算法参数与复现设置

为保证计算过程可复现，各阶段参数统一列于表2。差分进化在不同问题中使用固定随机种子重复搜索，并以严格几何模型验收候选；问题 5 的候选生成、组合精修和向后删除均使用同一组运动学与约束函数。

表 2 主要算法参数与复现设置

环节	参数设置
差分进化	问题 2 种群 44、迭代 65；问题 3 种群 56、迭代 90 并采用多随机种子；问题 5 候选生成种群 30、迭代 42
变异与交叉	缩放因子 0.75，交叉概率 0.85，每个试验向量至少继承一个变异维度
局部搜索	坐标正负方向轮换，连续一轮无改进时步长减半，按问题规模执行 2-5 轮
正式评价	圆周 24 点、高度 5 层，时间步长 0.02 s；问题 1-4 最终表值采用 0.01 s 复算
高密度复算	圆周 48 点、高度 9 层，时间步长 0.01 s；收敛上限采用圆周 72 点、高度 13 层
MILP 认证	20 套整航线候选，时间步长 0.05 s，整数规划相对最优性间隙设为 0

六、模型求解与结果

6.1 问题 1：给定策略的遮蔽时长

6.1.1 解析计算与严格判定

FY1 以 120 m/s 沿 x 轴负向飞行，故

$$\mathbf{u}_1(t) = (17800 - 120t, 0, 1800). \quad (24)$$

在 1.5 s 投放，得到

$$\mathbf{d}_{11} = (17620, 0, 1800). \quad (25)$$

延迟 3.6 s 起爆，起爆时刻为 5.1 s，起爆点为

$$x_b = 17620 - 120 \times 3.6 = 17188, \quad (26)$$

$$z_b = 1800 - \frac{1}{2} \times 9.8 \times 3.6^2 = 1736.496. \quad (27)$$

所以

$$\mathbf{b}_{11} = (17188, 0, 1736.496). \quad (28)$$

将导弹轨迹、云团下沉轨迹代入完整目标遮蔽模型，得到中心视线遮蔽时长 1.436 s，完整圆柱目标严格遮蔽时长为

$$\boxed{T_1 = 1.390 \text{ s}}. \quad (29)$$

6.1.2 结果解释与基准作用

该结果可由几何关系作交叉检查。烟幕云团自 5.1 s 开始生效，其有效期覆盖导弹 M1 到达假目标前的主要接近阶段；但云团中心仅竖直下沉，而导弹视线在水平面和高度方向持续变化，因此有效遮蔽只会出现在较短时间窗内。若只考察导弹到圆柱中心的单条视线，计算得到 1.436 s，比完整圆柱结果多 0.046 s。这一差值说明圆柱边缘点比中心点更早露出，验证了采用完整目标判据的必要性。投放点与起爆点均由解析式直接获得，数值程序仅承担视线距离和时间积分计算，因而问题 1 也构成后续代码的单元校验算例。

6.2 问题 2：单机单弹优化

6.2.1 决策变量与求解方法

决策变量取为

$$\mathbf{x} = (\theta_1, v_1, t_d, \tau), \quad (30)$$

满足

$$0 \leq \theta_1 < 360^\circ, \quad 70 \leq v_1 \leq 140, \quad t_d \geq 0, \quad \tau \geq 0, \quad z_b \geq 1. \quad (31)$$

对于给定 $(\theta_1, v_1, t_d, \tau)$ ，投放点和起爆点由运动方程唯一确定，故无需将空间坐标重复设为决策变量。这样既减少变量维数，又能从编码层面保证坐标满足动力学关系。搜索上界根据 M1 到达假目标的时间和烟幕 20 s 有效期设置，使起爆过晚、必然无法产生遮蔽的区域不进入种群。初始种群除随机个体外，还加入朝向假目标、朝向真目标和沿初始视线方向的若干启发式个体，以降低完全随机初始化落入零得分平台的概率。

严格目标函数不连续，单次随机搜索可能得到不同局部解。因此程序使用固定随机种子，并对代理得分前列个体统一作严格复算和坐标修正。航向角采用周期边界，速度和时序变量采用截断边界。最终解的起爆延迟落在下界 $\tau = 0$ ，这意味着在题目未规定最短引信延迟时，尽早形成云团能够延长与导弹视线的交叠时间；该现象属于模型边界解，而不是约束遗漏。

6.2.2 优化结果与边界解解释

先用差分进化搜索连续代理目标，再用完整目标函数进行坐标局部修正。所得结果如表3。

表 3 问题 2 最优投放参数

航向角/ $^\circ$	速度/(m/s)	投放时刻/s	起爆延迟/s	投放点	起爆点
4.942	139.385	0.893	0	(17924, 11, 1800)	(17924, 11, 1800)

投放后立即起爆对应 $\tau = 0$ 。题目未规定最小引信延迟，因此该边界解在本文模型

中可行。严格遮蔽时长为

$$T_1 = 4.580 \text{ s}, \quad (32)$$

较问题 1 提高约 229.5%。

从空间位置看，最优航向与 x 轴正向夹角仅 4.942° ，速度接近允许上限，说明 FY1 需要迅速向 M1 的早期视线附近推进。与问题 1 给定策略相比，优化并非简单地把投放点沿原航线前移，而是同时调整了横向位置与起爆时刻，使烟幕球在导弹观察圆柱边缘的关键时间段内具有更大的几何余量。

6.3 问题 3：单机三弹时序优化

6.3.1 时序参数化与联合目标

同一无人机三枚弹共享航向和速度。采用参数化

$$t_1^d = x_3, \quad t_2^d = x_3 + 1 + x_4, \quad t_3^d = x_3 + 2 + x_4 + x_5, \quad (33)$$

从编码层面保证投弹间隔不小于 1 s。为避免经验时间范围遗漏，分别搜索聚焦时间域、完整飞行时间域及三弹均衡候选，再按“总时长优先、近等价时有效弹数优先”的词典序规则选择。

与问题 2 相比，问题 3 的目标不能写成三枚弹独立时长之和。设第 i 枚烟幕弹单独产生的遮蔽时刻集合为 $\mathcal{I}_i$ ，则联合遮蔽时长为

$$T_1 = \mu\left(\bigcup_{i=1}^3 \mathcal{I}_i\right), \quad (34)$$

其中 $\mu(\cdot)$ 表示时间集合的测度。若两个云团在同一时段重复遮蔽，重叠部分只能计入一次；若两个云团分别覆盖圆柱的不同区域，则还可能共同形成任一单弹都无法完成的完整遮蔽。因此，优化过程中始终对三枚弹联合调用几何判定，而不是将三个单弹评价值相加。

时序编码中的非负增量 x_4, x_5 将不等式约束转化为天然可行的参数化形式，避免差分进化频繁产生投弹间隔不足的个体。三枚弹共享的航向和速度只编码一次，决策向量包含 2 个航线变量、3 个投放时序变量和 3 个起爆延迟变量。求解时先优化首枚弹形成基础遮蔽区间，再逐步加入后续烟幕弹，并对全部变量联合修正，以减少贪心加入造成的时序锁定。

6.3.2 优化结果与三弹方案比较

表 4 问题 3 投放策略

弹号	航向角/°	速度/(m/s)	投放时刻/s	起爆延迟/s	独立遮蔽/s
1	5.414	140.000	0	0	2.560
2	5.414	140.000	1.000	0	4.110
3	5.414	140.000	5.731	4.877	0

三枚弹联合严格遮蔽时长为

$$T_1 = 6.410 \text{ s}.$$

(35)

第三枚弹在当前总时长最优候选中未产生额外边际增益。另行搜索得到三弹均有效方案，但联合时长仅 5.910 s，因此正式结果保留总时长较大的 6.410 s 方案。该结果说明“使用三枚弹”不等价于“三枚弹均增加联合时长”，论文不将其解释为三弹均有效。

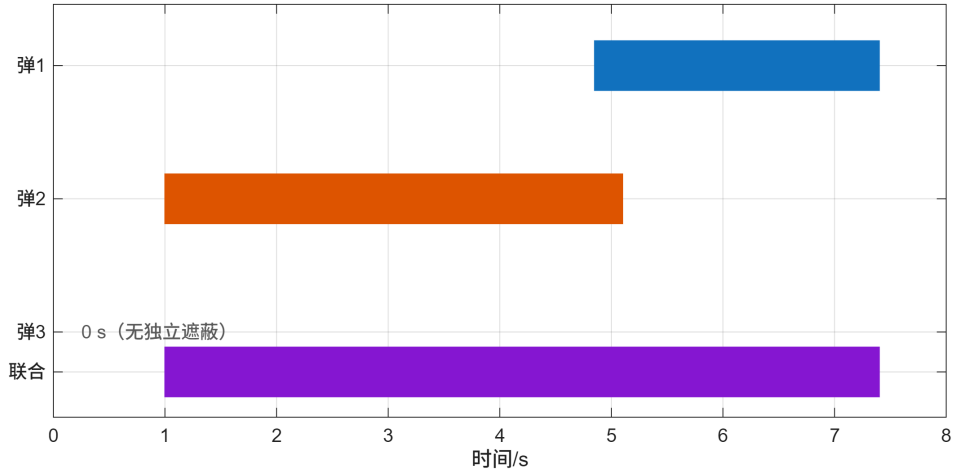


图 3 问题 3 单弹与联合方案的有效遮蔽区间对比

进一步比较可知，前两枚弹的独立遮蔽时长之和为 6.670 s，而联合时长为 6.410 s，两者相差 0.260 s，反映出两段遮蔽存在少量时间重叠。第三枚弹虽未增加严格联合时长，但其参数仍满足题目约束。若实际任务更强调资源利用率，可在总时长允许小幅下降的条件下增加“有效弹数”次级目标；本题以累计遮蔽时间最大为唯一主目标，因此不为形式上的三弹均有效牺牲 0.500 s 的主目标值。

6.4 问题 4：三机单弹协同优化

6.4.1 多平台候选组合与联合修正

三架无人机分别生成接近有效视线的单弹候选，再将 12 个连续变量联合局部修正。三枚烟幕弹的独立贡献及联合效果如图4所示。

三架无人机各自具有独立航向和速度，决策向量为

$$\mathbf{x} = (\theta_1, v_1, t_1^d, \tau_1, \theta_2, v_2, t_2^d, \tau_2, \theta_3, v_3, t_3^d, \tau_3). \quad (36)$$

若直接在 12 维空间随机搜索，绝大多数个体的云团远离有效视线。为提高搜索效率，先分别求解三个受限的单机单弹子问题，保留每架无人机的若干非支配候选；非支配标准同时考虑严格遮蔽时间和起爆点到目标视线的几何裕度。随后从三组候选中组合初值，并在联合目标下轮流调整各无人机的四个变量。每次调整都重新计算三团烟幕的并集覆盖，因而能够识别跨平台拼接遮蔽和时间重叠。

问题 4 不要求三枚烟幕弹平均分担任务。较早到达有效区域的 FY1 负责建立前段遮蔽，FY2 和 FY3 从不同方位补充后续视线。由于三架无人机的起点、高度和相对方位不同，各自最优速度与引信延迟差异明显，这也说明直接沿用 FY1 的单机策略并作平移并不合理。

6.4.2 协同策略与区间接力结果

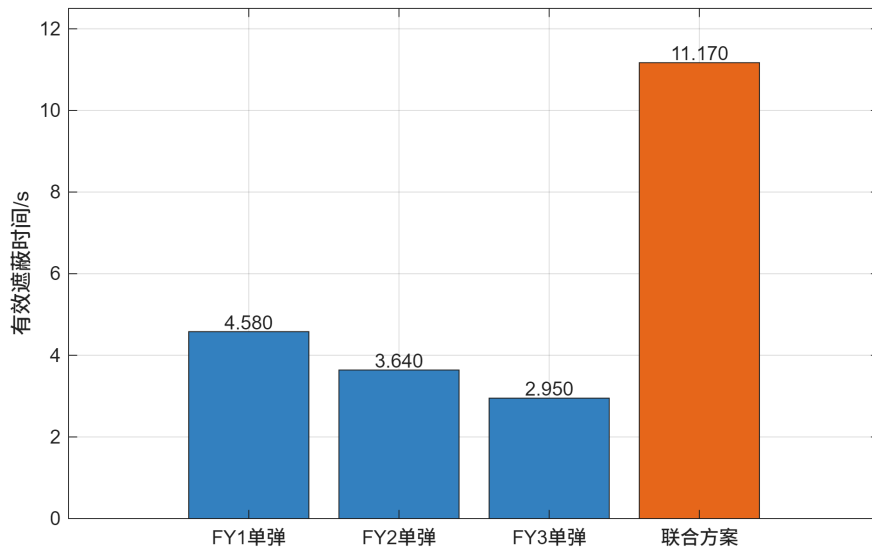


图 4 问题 4 单弹独立遮蔽时间与联合方案效果

表 5 问题 4 三架无人机投放策略

无人机	航向角/°	速度/(m/s)	投放/s	延迟/s	独立遮蔽/s
FY1	4.942	139.385	0.893	0	4.580
FY2	238.700	103.265	7.929	7.416	3.640
FY3	89.251	115.591	22.629	4.087	2.950

三弹独立时长之和为 11.170 s，联合严格遮蔽时间同样为

$$T_1 = 11.170 \text{ s}. \quad (37)$$

高密度复算为 11.150 s，差异仅 0.020 s，三架无人机均具有正边际贡献。

独立时长之和与联合时长在正式网格上均为 11.170 s，表明三个主要遮蔽区间基本首尾衔接而没有明显重复。高密度复算的 0.020 s 下降来自区间端点与圆柱表面采样加密，并未改变三机协同关系。相比问题 3，同样使用三枚烟幕弹时，问题 4 将联合时长提高 4.760 s，说明多平台从不同初始位置接近视线，比单平台受固定航迹约束连续投弹具有更大的空间自由度。

6.5 问题 5：五机多弹多目标协同优化

6.5.1 分层优化框架

问题 5 约含 40 个连续变量并伴随离散任务分配，直接联合搜索容易陷入大面积零值平台。本文采用以下步骤：

若五架无人机均投满三枚烟幕弹，除 10 个航线变量外，还需优化 30 个投放与延迟变量；同时，每枚弹可能服务 M1、M2、M3 中任一目标，朴素任务分配已有 3^{15} 种。更重要的是，同一云团可能在不同时间段服务不同导弹，预先固定“弹-导弹”一一对应关系会限制解空间。因此，任务标签只用于生成候选和解释结果，正式评价仍对每个有效云团与三枚导弹逐一进行几何判定。

分层算法将原问题拆成三个尺度：第一层确定无人机航线及其主要服务目标，解决大尺度空间接近问题；第二层优化同一航线上的投放时序，使云团落入导弹视线的有效时间窗；第三层以全部导弹累计遮蔽时间为目标联合微调，并通过边际贡献删除冗余弹。该分解不改变正式目标函数，只改变候选生成方式，能够把计算资源集中在可能产生严格遮蔽的局部区域。

- 1) 针对每个“无人机-导弹”组合生成三弹候选航线；
- 2) 枚举五架无人机的任务分配组合，以完整圆柱目标遮蔽时间进行筛选；
- 3) 对选中组合按无人机分块调整航向、速度、投放时刻和起爆延迟；
- 4) 计算删除边际贡献，在总时长和最短目标时长容差内删除冗余弹；
- 5) 提取当前遮蔽空缺，搜索新增弹，并将新增弹与对应航线联合优化；
- 6) 循环执行“增加-精修-验收”，无增益时停止，再进行一次最终冗余删除；
- 7) 以高密度完整目标模型复算，并用 0-1 整数规划认证候选集组合最优性。

正式接受规则为：若总时长提高超过 0.01 s 则接受；若总时长下降不超过 0.02 s，则仅在最短目标时长提高超过 0.01 s 时接受。该规则保证总遮蔽时长始终为主目标。

具体而言，记当前方案的评价向量为

$$f(S) = \left(\sum_{j=1}^3 T_j(S), \min_j T_j(S), -|S| \right). \quad (38)$$

算法以第一分量为主，第二分量用于主目标近等价时改善三枚导弹之间的平衡，第三分量用于删除不产生贡献的烟幕弹。这里没有把三个指标简单加权，是因为权重变化可能使“少量均衡收益”抵消“明显总时长损失”。采用带容差的词典序比较后，各层含义清晰，且与题目要求的总有效遮蔽时间尽可能长保持一致。

新增烟幕弹搜索并非在全部时间域盲目进行。程序先从当前遮蔽指标 $H_j(t)$ 中提取未覆盖区间，选择持续时间较长的空缺作为目标窗，再反推烟幕云团应接近的导弹-目标视线位置。对每个候选，联合调整对应无人机的航向、速度、新弹投放时刻及引信延迟；若新弹破坏同机已有弹的投放间隔，则重新排序时刻并投影到可行域。这样形成“从遮蔽缺口反推投放参数”的事件驱动搜索，比固定时间网格枚举具有更高效率。

6.5.2 最终航迹和投放策略

最终使用 9 枚烟幕弹。五架无人机航向与速度见表6，具体投放参数见表7。

表 6 问题 5 无人机航向与速度

无人机	航向角/°	速度/(m/s)
FY1	5.608	139.897
FY2	295.947	96.503
FY3	95.932	96.570
FY4	259.905	140.000
FY5	117.096	137.947

表 7 问题 5 烟幕弹投放与起爆参数

无人机	弹序	投放/s	延迟/s	投放点 $(x, y, z)/m$	起爆点 $(x, y, z)/m$	主要目标	独立时长/s
FY1	1	0	0	(17800,0,1800)	(17800,0,1800)	M1	2.560
FY1	2	1.000	0	(17939,14,1800)	(17939,14,1800)	M1	4.040
FY2	1	10.000	1.188	(12422,532,1400)	(12472,429,1393)	M2	2.800
FY2	2	11.343	0.021	(12479,416,1400)	(12480,414,1400)	M2	3.680
FY3	1	25.690	4.272	(5744,-532,700)	(5701,-122,611)	M3	2.500
FY3	2	30.000	3.750	(5701,-118,700)	(5663,242,631)	M2	2.460
FY4	1	0.866	10.968	(10979,1881,1800)	(10710,369,1210)	M2	3.480
FY4	2	2.250	12.000	(10945,1690,1800)	(10650,36,1094)	M1	3.540
FY5	1	12.780	0.280	(12197,-431,1300)	(12179,-396,1300)	M3	3.620

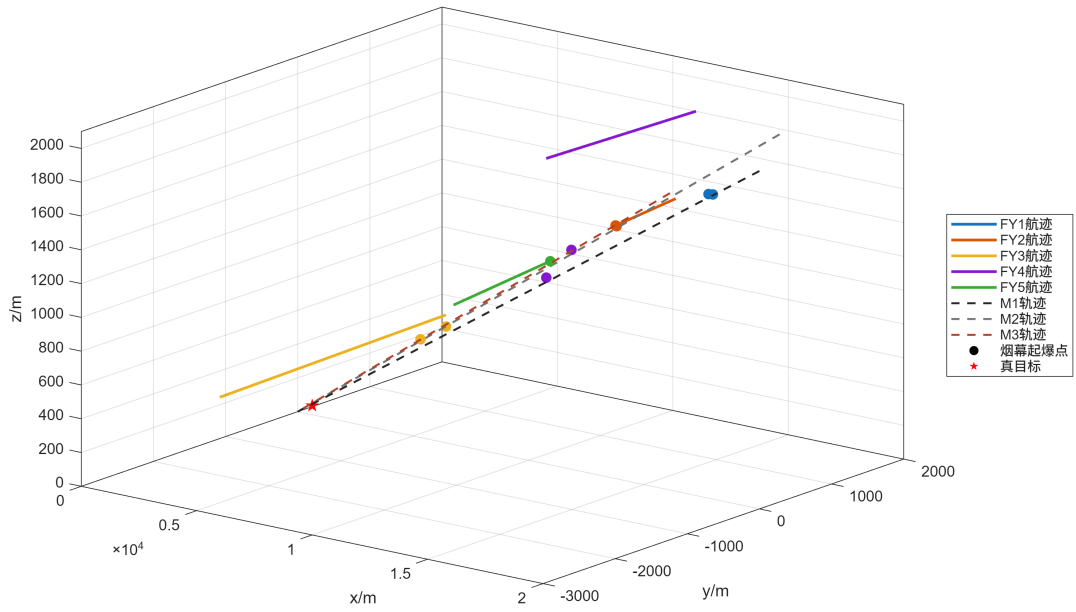


图 5 问题 5 无人机航迹、导弹轨迹与烟幕起爆点

6.5.3 遮蔽结果与区间分析

三枚导弹的严格遮蔽结果为

$$(T_1, T_2, T_3) = (9.960, 12.500, 6.120) \text{ s}, \quad (39)$$

总时长为

$$T_{\text{sum}} = 28.580 \text{ s}. \quad (40)$$

正式网格与高密度复算的分项目标值见表8。分项时长仅发生 0.01 s 量级的转移，总时长保持不变。

表 8 问题 5 不同评价精度下的遮蔽时长

评价口径	M1/s	M2/s	M3/s	总计/s
正式网格	9.960	12.500	6.120	28.580
高密度网格	9.960	12.490	6.130	28.580

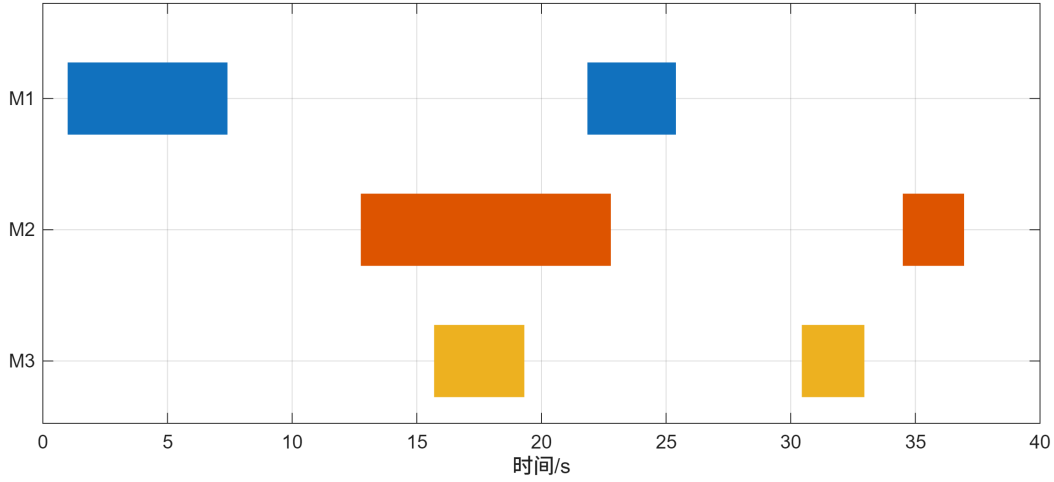


图 6 问题 5 三枚导弹在前 40 s 内的有效遮蔽区间

M1 在约 1.00 ~ 7.41 s 和 21.85 ~ 25.40 s 受到遮蔽；M2 在约 12.76 ~ 22.79 s 和 34.50 ~ 36.96 s 受到遮蔽；M3 在约 15.70 ~ 19.32 s 和 30.45 ~ 32.96 s 受到遮蔽。区间端点取相邻离散时刻的中点，因此各段长度之和与数值积分时长一致。各区间允许不连续，符合题意。M2 遮蔽时间较长是总时长优先目标下的自然结果；若任务更强调均衡，可在总时长容差内进一步最大化 $\min(T_1, T_2, T_3)$ 。

6.5.4 候选集整数规划认证

每架无人机保留当前正式航线及分别针对三枚导弹形成的三套候选，共 20 套整航线方案。设 x_p 表示是否选择方案 p ， y_{jn} 表示导弹 j 在离散时刻 t_n 是否完整遮蔽，依据 0-1 整数规划的标准建模方法 [4] 预计算点级遮蔽矩阵 a_{pjnq} ，建立

$$\max \quad \Delta t \sum_{j,n} y_{jn}, \quad (41)$$

$$\text{s.t.} \quad y_{jn} \leq \sum_p a_{pjnq} x_p, \quad \forall j, n, q, \quad (42)$$

$$\sum_{p \in \mathcal{P}_k} x_p \leq 1, \quad k = 1, \dots, 5, \quad (43)$$

$$x_p, y_{jn} \in \{0, 1\}. \quad (44)$$

在 $\Delta t = 0.05$ s 的认证网格下，使用 MATLAB 的 `intlinprog` 求解 [5]，最优性间隙为 0，选择的正是当前五架无人机正式航线。离散目标值为 28.700 s，高密度严格复算为 28.580 s。二者的 0.120 s 差异来自 0-1 网格端点累计。该结果证明当前方案是候选库与认证离散精度下的全局最优组合，不构成原连续 40 维模型的全局最优证明。

七、 结果分析与讨论

7.1 五问结果的递进比较

表9汇总五个问题的资源规模与严格遮蔽结果。问题 1 到问题 2 只增加四个连续决策自由度，遮蔽时间由 1.390 s 增至 4.580 s，表明给定策略与最佳视线交会位置仍有较大差距。问题 3 增加投弹资源后提升至 6.410 s，但受单一航迹和 1 s 投弹间隔限制，新增烟幕弹的边际收益递减。问题 4 保持三枚弹不变而改用三架无人机，结果达到 11.170 s，说明平台初始位置与航向自由度是扩大遮蔽时间窗的重要因素。问题 5 同时面向三枚导弹，总时长达到 28.580 s，平均每枚导弹获得 9.527 s 的完整遮蔽。

表 9 五个问题的资源规模与结果比较

问题	无人机数	使用烟幕弹数	干扰目标	累计遮蔽时间/s
1	1	1	M1	1.390
2	1	1	M1	4.580
3	1	3	M1	6.410
4	3	3	M1	11.170
5	5	9	M1–M3	28.580

上述比较还说明，烟幕弹数量不是决定效果的唯一因素。问题 3 的第三枚弹在总时长最优解中边际贡献为零，而问题 4 的三枚弹均产生正贡献；差异来自云团能否在空间和时间上覆盖不同的视线缺口。因此，策略设计应优先保证航迹与视线的几何匹配，再讨论投弹数量。若先固定航线后平均安排投弹时刻，容易产生多个云团重复覆盖同一时段，造成资源浪费。

7.2 问题 5 的时空协同机理

问题 5 的九枚有效烟幕弹形成“前段快速遮蔽、中段多方向接力、后段补充空缺”的时序结构。FY1 在任务开始后的 1 s 内连续投放两枚弹，主要覆盖 M1 的早期接近阶段；FY2 和 FY4 在中段从不同方向靠近 M2 视线，形成约 12.76–22.79 s 的主要遮蔽区间；FY3 和 FY5 则利用较晚的起爆时刻补充 M2、M3 后段区间。由于三枚导弹初始位置和到达时间不同，最优方案没有强行采用统一投弹节奏，而是让各无人机根据视线移动速度和剩余飞行时间分工。

从航线参数看，FY1 与 FY3 分别接近速度上限和中等速度，FY4 达到 140 m/s 上界，FY2 速度相对较低。速度取值差异并不意味着低速平台利用不足：较低速度可以延迟到达特定视线位置，使起爆时刻与导弹经过目标方向的时间窗相匹配。类似地，引信延迟从 0 到 12 s 均有出现，说明起爆点不是由投放点邻域简单决定，而是由无人机水平运动、烟幕弹自由落体和导弹视线变化共同决定。

三枚导弹的遮蔽时长并不完全均衡，M3 的 6.120 s 低于 M1 和 M2。原因一是 M3 的初始距离较短，可利用的飞行时间窗更窄；原因二是总时长优先的目标会把资源分配给单位投弹边际收益更高的 M1、M2。若任务要求三枚导弹获得相近保护，可改用最大化 $\min_j T_j$ 或在 T_j 上设置最低保障约束，但这将形成与当前题意不同的优化目标，并可能降低总遮蔽时间。

7.3 全局最优性结论的适用边界

差分进化、坐标搜索和增量投弹共同给出一个高质量连续可行解，但这些启发式步骤本身不能证明 40 维混合连续模型的全局最优。本文的严格结论来自后续候选集整数规划：在 20 套完整航线候选、0.05 s 时间网格和既定目标采样下，分支定界返回 0 最优性间隙，故不存在更优的候选组合。该结论对“候选集中的组合选择”是全局性的，对“所有可能航向、速度和时序”则不是全局性的。

这一表述比直接宣称连续原问题全局最优更为严谨。高密度复算保持 28.580 s，说明当前方案对数值离散具有稳定性；多随机种子搜索和局部修正多次回到相近结果，则提供经验上的近优证据。若需要连续意义下的严格证明，还需对航向、速度和时序变量构造区间上界，并使用空间分支定界逐盒排除，但计算量将远高于本题规模下的候选集认证。

八、模型检验与敏感性分析

8.1 运动学和约束检验

独立验证程序逐项检查速度、投弹间隔、起爆高度和数据有限性。问题 5 各架无人机投弹数为 2、2、2、2、1，最小投弹间隔分别为 1.000 s、1.343 s、4.311 s、1.384 s；最低起爆高度为 610.594 m。全部显式约束均满足。

8.2 时间与空间离散收敛

采用圆周 24 点、高度 5 层及 $\Delta t = 0.04$ s 作为粗网格，采用圆周 72 点、高度 13 层及 $\Delta t = 0.01$ s 作为精细网格。结果见表10和图7。

表 10 离散精度收敛结果

问题	粗网格/s	精细网格/s	绝对差/s
1	1.400	1.390	0.010
2	4.600	4.580	0.020
3	6.440	6.410	0.030
4	11.200	11.150	0.050
5	28.520	28.580	0.060

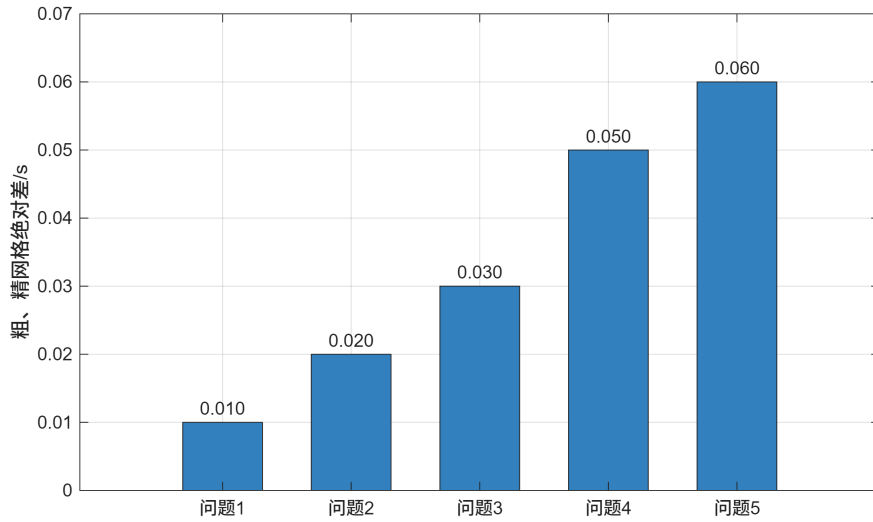


图 7 粗、精网格遮蔽时间的绝对差异

问题 5 高密度结果为 (9.960, 12.490, 6.130) s，总时长仍为 28.580 s。图7直接绘制粗、精网格的绝对差，避免两组总时长曲线因尺度接近而相互遮盖。问题 4 的 11.150 s 是固定策略在最密时间与空间网格组合下的复算值，11.170 s 是正式评价网格下的提交值，二者相差 0.020 s；表10中的 0.050 s 比较的是预设粗、精两端网格，口径不同。各问随网格加密趋于稳定，正式结果保留三位小数具有合理精度。

8.3 执行误差敏感性分析

为区分数值收敛与实际执行稳健性，对问题 5 进行 100 次随机扰动。为控制重复计算量，该实验采用圆周 24 点、高度 5 层及 0.04 s 时间步长，对应未扰动基准值 28.520 s。每架无人机航向误差取 $[-0.2^\circ, 0.2^\circ]$ ，速度相对误差取 $[-1\%, 1\%]$ 并截断到允许范围；同一无人机的投放时刻整体偏移取 $[-0.05, 0.05]$ s，引信延迟误差取 $[-0.05, 0.05]$ s。整体平移投放时刻可保持同机投弹间隔不变。结果见表11。

表 11 问题 5 执行误差敏感性分析结果

统计量	M1/s	M2/s	M3/s	总计/s
基准值	9.960	12.480	6.080	28.520
扰动均值	8.632	7.923	3.340	19.895
标准差	1.404	2.057	1.573	3.249
最小值	6.320	1.320	0.000	9.760
最大值	10.000	12.960	5.960	27.160

离散网格加密时总时长保持稳定，但小幅执行误差会显著降低严格完整遮蔽时长，尤其 M2 和 M3 较为敏感。原因是完整目标判据要求所有可见采样点同时被覆盖，烟幕球靠近可见轮廓边界时会产生明显的 0-1 跳变。因此，当前方案适合作为理想条件下的

性能上界和投放基准；实际部署应将期望时长、最坏情形时长或扰动成功率纳入优化目标，而不能仅追求名义参数下的最大值。

图8进一步给出 100 次扰动样本的总时长分布。样本明显集中在名义基准值左侧，说明性能下降不是由个别极端样本造成，而是航向和时序误差共同作用下的系统性结果。

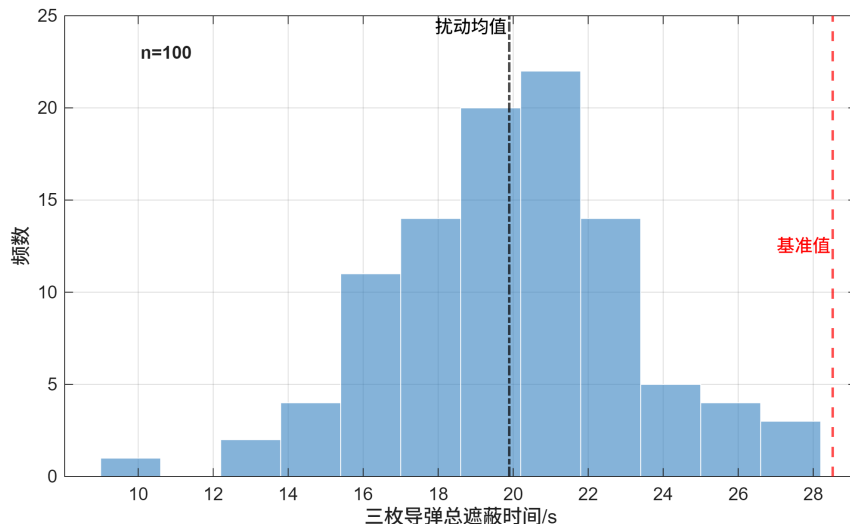


图 8 问题 5 执行误差扰动下的总遮蔽时间分布

8.4 边际贡献检验

对正式方案中的每枚烟幕弹计算

$$\Delta T_{ij} = T_j(S) - T_j(S \setminus \{i\}). \quad (45)$$

问题 5 保留的 9 枚弹在高密度模型下均存在正边际贡献，说明最终后向删除已排除可在数值容差内无损移除的冗余弹。独立遮蔽时间用于填写结果表，边际贡献仅用于判断冗余和确定主要干扰目标，避免混淆两类指标。

九、模型评价与改进方向

9.1 几何真实性与计算代价

运动学关系均采用显式表达式，投放点、起爆点和云团轨迹能够由原始决策变量直接复核。完整圆柱判据利用了真目标的尺寸和可见表面信息，并允许不同云团覆盖不同目标点，较中心视线判定更严格。相应代价是每个时刻都要遍历导弹、云团和表面采样点，计算量随时间和空间网格加密迅速增加。现有模型还忽略风场、空气阻力和云团扩散，实际烟幕可能出现水平漂移和形变。后续可在不改变几何框架的基础上，将球形云团替换为随时间变化的椭球或随机集合。

9.2 搜索效率与最优性边界

连续代理先将候选引入有效视线附近，严格模型再负责验收，避免差分进化长期停留在零值平台；问题 5 的缺口驱动增量搜索则把计算资源集中于尚未遮蔽的时间段。高密度复算、边际贡献和整数规划构成了由可行性到组合最优性的验证链。不过，差分进化与坐标搜索仍属于启发式方法，MILP 的零间隙结论也只适用于 20 套候选航线和给定离散网格。扩大候选库、增加独立随机种子并构造连续变量的区间上界，可进一步缩小当前解与原连续模型全局最优值之间的不确定范围。

9.3 名义性能与执行稳健性

名义参数下问题 5 达到 28.580 s，且时间和空间网格加密没有改变总时长；但执行误差模拟均值下降到 19.895 s，说明几何边界附近的高性能方案不一定具有良好容错性。实际应用中可将期望遮蔽时间、低分位数时长或最坏情形时长作为目标，并显式加入航向、速度和时序误差。对于问题 2 的零延迟边界解和问题 3 的零边际第三枚弹，也应结合实际引信能力与资源利用要求给出备选策略，而不能只依据名义总时长作出部署选择。

十、 结论

本文以导弹、无人机、烟幕弹和云团的统一运动关系为基础，构造了面向圆柱可见表面的严格联合遮蔽判据。连续接近度代理解决了严格目标的大面积零值问题，缺口驱动增量投弹与冗余删除则用于控制多平台方案规模。

问题 1 的严格遮蔽时长为 1.390 s；单机单弹优化将其提高到 4.580 s；单机三弹和三机单弹方案分别达到 6.410 s 和 11.170 s。问题 5 最终保留 9 枚具有正边际贡献的烟幕弹，对 M1、M2、M3 分别遮蔽 9.960 s、12.500 s 和 6.120 s，总时长为 28.580 s。结果显示，多平台从不同方向接近导弹视线，比沿固定航迹简单增加投弹数量更能延长联合遮蔽时间。

全部方案均通过速度、投弹间隔和起爆高度检查。高密度复算保持问题 5 总时长不变，候选集整数规划在 0.05 s 网格下返回零最优性间隙，因此当前方案在候选库和给定离散精度内达到全局最优组合。执行扰动试验同时揭示了名义性能与稳健性之间的差异：若用于实际部署，还需围绕航向和时序误差重新优化，而不能直接将名义最优参数作为唯一方案。

参考文献

- [1] Storn R, Price K. Differential evolution – A simple and efficient heuristic for global optimization over continuous spaces[J]. Journal of Global Optimization, 1997, 11(4): 341–359.
- [2] de Berg M, Cheong O, van Kreveld M, Overmars M. Computational Geometry: Algorithms and Applications[M]. 3rd ed. Springer, 2008.

- [3] Burden R L, Faires J D. Numerical Analysis[M]. 9th ed. Brooks/Cole, 2011.
- [4] Taha H A. Operations Research: An Introduction[M]. 10th ed. Pearson, 2017.
- [5] MathWorks. Optimization Toolbox Documentation: intlinprog[EB/OL]. <https://www.mathworks.com/help/optim/ug/intlinprog.html>, 2026-08-30.

附录 A 主要算法伪代码

1.1 完整目标遮蔽判定

```

1  for each missile j
2      for each time t
3          find active smoke clouds
4          find visible points on the cylinder
5          for each visible point q
6              covered(q) = any(distance(cloud, segment(Mj,q)) <= 10)
7          end
8          H(t) = all(covered)
9      end
10     T(j) = trapz(t,H)
11 end

```

1.2 问题 5 增量优化流程

```

1  plan = initial_candidate_plan;
2  plan = block_refine(plan);
3  plan = remove_redundant_bombs(plan);
4
5  while bomb_count < 15
6      candidate = search_new_bomb(plan);
7      candidate = block_refine(candidate);
8      if total_gain > 0.01 || ...
9          (total_gain >= -0.02 && min_gain > 0.01)
10         plan = candidate;
11     else
12         break
13     end
14 end
15
16 plan = remove_redundant_bombs(plan);

```

1.3 候选集整数规划

```

1 for each UAV plan p
2     precompute point_cover(p, missile, time, target_point)
3 end
4
5 % x: selected UAV plans, y: fully covered missile-time nodes
6 maximize sum(dt .* y)
7 subject to
8     y(j,n) <= sum(point_cover(p,j,n,q) * x(p)), for all q
9     sum(x(plans_of_uav_k)) <= 1
10    x, y are binary

```

附录 B 完整 MATLAB 源程序

以下程序与支撑材料中的源文件保持一致，可由 `run_all.m` 顺序复现五问结果、验证数据、敏感性分析、整数规划认证和论文图像。

Listing 1 一键运行程序 `run_all.m`

```

1 clear; clc;
2 base_dir = fileparts(mfilename('fullpath'));
3 addpath(base_dir);
4 old_visible = get(0,'DefaultFigureVisible');
5 set(0,'DefaultFigureVisible','off');
6 cleanup = onCleanup(@() set(0,'DefaultFigureVisible',old_visible));
7 fprintf('smoke plan\n');
8 run(fullfile(base_dir,'solve_problem1.m'));
9 run(fullfile(base_dir,'solve_problem2.m'));
10 run(fullfile(base_dir,'solve_problem3.m'));
11 run(fullfile(base_dir,'solve_problem4.m'));
12 run(fullfile(base_dir,'solve_problem5.m'));
13 export_results();
14 validate_results();
15 convergence_check();
16 sensitivity_check();
17 solve_problem5_milp(.05);
18 run(fullfile(base_dir,'..','paper','make_figures.m'));
19 fprintf('done\n');

```

Listing 2 题目数据 `data_a.m`

```

1 function D = data_a(na,nz)
2 if nargin<1, na=24; end
3 if nargin<2, nz=5; end
4 D.g = 9.8;
5 D.vm = 300;
6 D.rs = 10;

```



```

7 D.vs = 3;
8 D.life = 20;
9 D.m0 = [20000 0 2000; 19000 600 2100; 18000 -600 1900];
10 D.u0 = [17800 0 1800; 12000 1400 1400; 6000 -3000 700; ...
11         11000 2000 1800; 13000 -2000 1300];
12 D.T = vecnorm(D.m0, 2, 2) / D.vm;
13 D.tc = [0 200 5];
14
15 a = linspace(0,2*pi,na+1)';
16 a(end) = [];
17 z = linspace(0,10,nz)';
18 [aa, zz] = meshgrid(a, z);
19 D.side = [7*cos(aa(:)), 200+7*sin(aa(:)), zz(:)];
20 D.normal = [cos(aa(:)), sin(aa(:)), zeros(numel(aa),1)];
21
22 r = [0 3.5 7];
23 top = [0 200 10];
24 for k = 2:numel(r)
25     top = [top; r(k)*cos(a), 200+r(k)*sin(a), 10*ones(numel(a),1)]; %#ok<
26         AGROW>
27 end
28 D.top = top;
29 end

```

Listing 3 烟幕弹轨迹 make_clouds.m

```

1 function C = make_clouds(routes, bombs, D)
2 n = size(bombs,1);
3 C.drop = nan(n,3);
4 C.burst = nan(n,3);
5 C.tb = nan(n,1);
6 C.te = nan(n,1);
7 C.ok = size(routes,2) >= 2 && size(bombs,2) == 3;
8
9 if ~C.ok || isempty(bombs), return; end
10 kall = bombs(:,1);
11 C.ok = all(isfinite(bombs),'all') && all(kall == round(kall)) ...
12     && all(kall >= 1 & kall <= size(routes,1));
13 if ~C.ok, return; end
14
15 used = unique(kall)';
16 for k = used
17     th = routes(k,1); v = routes(k,2);
18     td = sort(bombs(kall==k,2));
19     C.ok = C.ok && all(isfinite([th v])) && th >= 0 && th <= 360 ...
20         && (numel(td) < 2 || all(diff(td) >= 1-1e-9));
21 end

```

```

22 if ~C.ok, return; end
23
24 for i = 1:n
25     k = bombs(i,1);
26     td = bombs(i,2);
27     tau = bombs(i,3);
28     th = routes(k,1);
29     v = routes(k,2);
30     e = [cosd(th), sind(th), 0];
31     tb = td + tau;
32     C.drop(i,:) = D.u0(k,:) + v*td*e;
33     C.burst(i,:) = D.u0(k,:) + v*tb*e - [0 0 0.5*D.g*tau^2];
34     C.tb(i) = tb;
35     C.te(i) = min(tb + D.life, tb + C.burst(i,3)/D.vs);
36     C.ok = C.ok && td >= 0 && tau >= 0 && v >= 70 && v <= 140 ...
37         && C.burst(i,3) >= 1 && all(isfinite(C.burst(i,:)));
38 end
39 end

```

Listing 4 严格遮蔽时间 coverage_time.m

```

1 function [dur,trace] = coverage_time(routes, bombs, mids, D, dt,
    full_target)
2 C = make_clouds(routes, bombs, D);
3 dur = zeros(numel(mids),1);
4 trace = struct('mid',{},'t',{},'hit',{'});
5 if ~C.ok, dur(:) = -1; return; end
6
7 for h = 1:numel(mids)
8     j = mids(h);
9     t = (0:dt:D.T(j))';
10    e = -D.m0(j,:) / norm(D.m0(j,:));
11    M = D.m0(j,:) + D.v*m*t.*e;
12    hit = false(numel(t),1);
13
14    if ~full_target
15        q = D.tc;
16        for i = 1:numel(C.tb)
17            id = find(t >= C.tb(i) & t <= C.te(i));
18            if isempty(id), continue; end
19            cc = C.burst(i,:) - [zeros(numel(id),2), D.vs*(t(id)-C.tb(i))];
20            hit(id) = hit(id) | line_hit(M(id,:), q, cc, D.rs);
21        end
22    else
23        for n = 1:numel(t)
24            active = find(t(n) >= C.tb & t(n) <= C.te);
25            if isempty(active), continue; end

```

```

26         m = M(n,:);
27         vis = sum(D.normal .* (m-D.side), 2) >= 0;
28         if m(3) > 10
29             q = [D.side(vis,:); D.top];
30         else
31             q = D.side(vis,:);
32         end
33         covered = false(size(q,1),1);
34         for i = active'
35             cc = C.burst(i,:) - [0 0 D.vs*(t(n)-C.tb(i))];
36             covered = covered | line_hit(repmat(m,size(q,1),1), q, ...
37                 repmat(cc,size(q,1),1), D.rs);
38         end
39         hit(n) = all(covered);
40     end
41 end
42 dur(h) = trapz(t, double(hit));
43 if nargout>1
44     trace(h).mid = j;
45     trace(h).t = t;
46     trace(h).hit = hit;
47 end
48 end
49 end
50
51 function yes = line_hit(m, q, c, r)
52 if size(q,1) == 1, q = repmat(q,size(m,1),1); end
53 v = q-m;
54 w = c-m;
55 u = sum(w.*v,2) ./ max(sum(v.^2,2), eps);
56 u = min(1,max(0,u));
57 d2 = sum((c-(m+u.*v)).^2,2);
58 yes = d2 <= r^2;
59 end

```

Listing 5 连续代理目标 coverage_score.m

```

1 function s = coverage_score(routes,bombs,mids,D,dt)
2 C = make_clouds(routes,bombs,D);
3 if ~C.ok, s = -1; return; end
4 s = 0;
5 for j = mids
6     t = (0:dt:D.T(j))';
7     e = -D.m0(j,:)/norm(D.m0(j,:));
8     M = D.m0(j,:) + D.vm*t.*e;
9     best = zeros(size(t));
10    for i = 1:numel(C.tb)

```

```

11     id = find(t>=C.tb(i) & t<=C.te(i));
12     if isempty(id), continue; end
13     cc = C.burst(i,:) - [zeros(numel(id),2),D.vs*(t(id)-C.tb(i))];
14     q = repmat(D.tc,numel(id),1);
15     v = q-M(id,:);
16     u = sum((cc-M(id,:)).*v,2)./sum(v.^2,2);
17     u = min(1,max(0,u));
18     d = vecnorm(cc-(M(id,:)+u.*v),2,2);
19     best(id) = max(best(id),1./(1+exp((d-D.rs)/2)));
20 end
21 s = s + trapz(t,best);
22 end
23 end

```

Listing 6 遮蔽区间 coverage_intervals.m

```

1 function info = coverage_intervals(routes,bombs,mids,D,dt)
2 [dur,tr] = coverage_time(routes,bombs,mids,D,dt,true);
3 info = struct([]);
4 for h = 1:numel(mids)
5     t=tr(h).t; hit=tr(h).hit;
6     info(h).mid=mids(h);
7     info(h).duration=dur(h);
8     info(h).covered=runs(t,hit);
9     info(h).gaps=runs(t,~hit);
10 end
11 end
12
13 function a = runs(t,x)
14 d = diff([false;x>false]);
15 i1 = find(d(1:end-1)==1);
16 i2 = find(d(2:end)==-1);
17 left = t(i1); right = t(i2);
18 q = i1>1;
19 left(q) = (t(i1(q)-1)+t(i1(q)))/2;
20 q = i2<numel(t);
21 right(q) = (t(i2(q))+t(i2(q)+1))/2;
22 a = [left,right];
23 end

```

Listing 7 差分进化 de_search.m

```

1 function [best, fbest] = de_search(fun, lb, ub, np, ng, seed)
2 rng(seed);
3 d = numel(lb);
4 pop = lb + rand(np,d).*(ub-lb);
5 fit = zeros(np,1);

```

```

6  for i = 1:np, fit(i) = fun(pop(i,:)); end
7
8  for g = 1:ng
9      for i = 1:np
10         ids = randperm(np,3);
11         while any(ids == i), ids = randperm(np,3); end
12         y = pop(ids(1),:) + 0.75*(pop(ids(2),:)-pop(ids(3),:));
13         y = min(ub,max(lb,y));
14         take = rand(1,d) < 0.85;
15         take(randi(d)) = true;
16         z = pop(i,:);
17         z(take) = y(take);
18         fz = fun(z);
19         if fz < fit(i), pop(i,:) = z; fit(i) = fz; end
20     end
21 end
22 [fbest,k] = min(fit);
23 best = pop(k,:);
24 end

```

Listing 8 坐标局部搜索 local_search.m

```

1  function [x, fx] = local_search(fun, x, lb, ub, step, rounds)
2  fx = fun(x);
3  for r = 1:rounds
4      changed = false;
5      for j = 1:numel(x)
6          for s = [-1 1]
7              y = x;
8              y(j) = min(ub(j),max(lb(j),x(j)+s*step(j)));
9              fy = fun(y);
10             if fy < fx, x = y; fx = fy; changed = true; end
11         end
12     end
13     if ~changed, step = step/2; end
14 end
15 end

```

Listing 9 新增烟幕弹搜索 add_bombs.m

```

1  function [b,added] = add_bombs(r,b,D)
2  added=zeros(0,3);
3  base=coverage_time(r,b,1:3,D,.04,true);
4  trial=false;
5  while size(b,1)<15
6      pool=zeros(0,4);
7      for k=1:5

```

```

8      if sum(b(:,1)==k)>=3, continue; end
9      tm=sqrt(2*D.u0(k,3)/D.g);
10     old=b(b(:,1)==k,2);
11     for td=0:2:max(D.T)
12         if any(abs(old-td)<1), continue; end
13         for tau=0:1:tm
14             bb=[b;k td tau];
15             C=make_clouds(r,bb,D);
16             if ~C.ok || C.tb(end)>max(D.T), continue; end
17             sc=coverage_score(r,bb,1:3,D,.2);
18             pool(end+1,:)= [k td tau sc]; %#ok<AGROW>
19         end
20     end
21 end
22 if isempty(pool), break; end
23 [~,ix]=sort(pool(:,4),'descend');
24 pool=pool(ix(1:min(12,end)),:);
25 best=base; pick=[];
26 for i=1:size(pool,1)
27     bb=[b;pool(i,1:3)];
28     s=coverage_time(r,bb,1:3,D,.04,true);
29     if sum(s)>sum(best)+.01 || (sum(s)>=sum(best)-.02 && min(s)>min(
        best)+.01)
30         best=s; pick=pool(i,1:3);
31     end
32 end
33 if isempty(pick)
34     if trial, break; end
35     pick=pool(1,1:3); trial=true;
36 end
37 b=[b;pick]; %#ok<AGROW>
38 added=[added;pick]; %#ok<AGROW>
39 base=best;
40 if trial, break; end
41 end
42 end

```

Listing 10 候选点级遮蔽 plan_point_cover.m

```

1 function [t,A] = plan_point_cover(routes,bombs,j,D,dt)
2 C=make_clouds(routes,bombs,D);
3 t=(0:dt:D.T(j))';
4 q=[D.side;D.top];
5 A=false(numel(t),size(q,1));
6 e=-D.m0(j,:)/norm(D.m0(j,:));
7 M=D.m0(j,:)+D.vm*t.*e;
8 ns=size(D.side,1);

```

```

9  for n=1:numel(t)
10     m=M(n,:);
11     vis=[sum(D.normal.*(m-D.side),2)>=0;true(size(D.top,1),1)];
12     if m(3)<=10, vis(ns+1:end)=false; end
13     hit=~vis;
14     active=find(t(n)>=C.tb & t(n)<=C.te);
15     for i=active'
16         cc=C.burst(i,:)-[0 0 D.vs*(t(n)-C.tb(i))];
17         hit=hit|line_cover(m,q,cc,D.rs);
18     end
19     A(n,:)=hit;
20 end
21 end
22
23 function yes = line_cover(m,q,c,r)
24 v=q-m;
25 w=c-m;
26 u=sum(w.*v,2)./max(sum(v.^2,2),eps);
27 u=min(1,max(0,u));
28 yes=sum((c-(m+u.*v)).^2,2)<=r^2;
29 end

```

Listing 11 问题 1 求解 solve_problem1.m

```

1  base_dir = fileparts(mfilename('fullpath'));
2  result_dir = fullfile(base_dir,'..','result');
3  if ~exist(result_dir,'dir'), mkdir(result_dir); end
4
5  D = data_a();
6  routes = zeros(5,2);
7  routes(1,:) = [180 120];
8  bombs = [1 1.5 3.6];
9  C = make_clouds(routes,bombs,D);
10
11  t_center = coverage_time(routes,bombs,1,D,0.002,false);
12  t_full = coverage_time(routes,bombs,1,D,0.01,true);
13  save(fullfile(result_dir,'problem1.mat'),'routes','bombs','C','t_center','t_full');
14  fprintf('P1: %.3f s, full %.3f s\n',t_center,t_full);

```

Listing 12 问题 2 求解 solve_problem2.m

```

1  base_dir = fileparts(mfilename('fullpath'));
2  result_dir = fullfile(base_dir,'..','result');
3  if ~exist(result_dir,'dir'), mkdir(result_dir); end
4
5  D = data_a();

```

```

6  uav = 1; mid = 1;
7  tau_max = sqrt(2*D.u0(uav,3)/D.g);
8  lb = [0 70 0 0];
9  ub = [360 140 D.T(mid) tau_max];
10 f = @(x) cost_one(x,uav,mid,D,0.05,false);
11 [x,~] = de_search(f,lb,ub,44,65,102);
12 ff = @(z) cost_one(z,uav,mid,D,0.03,true);
13 [x,~] = local_search(ff,x,lb,ub,[8 6 1 0.8],5);
14 [routes,bombs] = decode_one(x,uav);
15 C = make_clouds(routes,bombs,D);
16 t_center = coverage_time(routes,bombs,mid,D,0.005,false);
17 t_full = coverage_time(routes,bombs,mid,D,0.01,true);
18 save(fullfile(result_dir,'problem2.mat'),'x','routes','bombs','C','t_center',
    't_full');
19 fprintf('P2: %.3f s, full %.3f s\n',t_center,t_full);
20
21 function y = cost_one(x,uav,mid,D,dt,full_target)
22 [routes,bombs] = decode_one(x,uav);
23 C = make_clouds(routes,bombs,D);
24 if ~C.ok || C.tb > D.T(mid), y = 1e4; return; end
25 y = -coverage_time(routes,bombs,mid,D,dt,full_target);
26 end
27
28 function [routes,bombs] = decode_one(x,uav)
29 routes = zeros(5,2);
30 routes(uav,:) = x(1:2);
31 bombs = [uav x(3) x(4)];
32 end

```

Listing 13 问题 3 求解 solve_problem3.m

```

1  base = fileparts(mfilename('fullpath'));
2  out = fullfile(base,'..','result');
3  if ~exist(out,'dir'), mkdir(out); end
4
5  D = data_a();
6  lb = [0 70 0 0 0 0 0 0];
7  tm = sqrt(2*D.u0(1,3)/D.g);
8  ub = [360 140 35 15 15 tm tm tm];
9  ub_wide = [360 140 D.T(1)-2 D.T(1)-2 D.T(1)-2 tm tm tm];
10
11 x_total = search_plan(false,lb,ub,D,203:207);
12 x_even = search_plan(true,lb,ub,D,303:305);
13 x_wide = search_plan(false,lb,ub_wide,D,403:405);
14 [~,~,~,t1,s1] = assess(x_total,D);
15 [~,~,~,t2,s2] = assess(x_even,D);
16 [~,~,~,t3,s3] = assess(x_wide,D);

```



```

17 x = x_total;
18 if t3>t1 || (t3>=t1-.05 && min(s3)>min(s1)), x=x_wide; t1=t3; s1=s3; end
19 if t2>=t1-.05 && min(s2)>min(s1), x=x_even; end
20
21 [routes,bombs,C,t_full,single_time,t_center,marginal_time] = assess(x,D);
22 save(fullfile(out,'problem3.mat'),'x','x_total','x_even','x_wide','routes',
    'bombs', ...
23     'C','t_center','t_full','single_time','marginal_time');
24 fprintf('P3: %.3f s, full %.3f s\n',t_center,t_full);
25
26 function x = search_plan(even,lb,ub,D,seeds)
27 f = @(z) score_cost(z,D,even);
28 [x,best] = de_search(f,lb,ub,56,90,seeds(1));
29 for seed = seeds(2:end)
30     [z,fz] = de_search(f,lb,ub,56,90,seed);
31     if fz<best, x=z; best=fz; end
32 end
33 g = @(z) exact_cost(z,D,false);
34 [x,~] = local_search(g,x,lb,ub,[8 6 1 1 1 .7 .7 .7],5);
35 h = @(z) exact_cost(z,D,true);
36 [x,~] = local_search(h,x,lb,ub,[8 6 1 1 1 .7 .7 .7],4);
37 end
38
39 function y = score_cost(x,D,even)
40 [r,b] = decode(x);
41 C = make_clouds(r,b,D);
42 if ~C.ok || any(C.tb>D.T(1)), y=1e4; return; end
43 s = coverage_score(r,b,1,D,.1);
44 if even
45     one=zeros(3,1);
46     for i=1:3, one(i)=coverage_score(r,b(i,:),1,D,.1); end
47     s=s+.25*sum(one)+.15*min(one);
48 end
49 y=-s;
50 end
51
52 function y = exact_cost(x,D,full)
53 [r,b] = decode(x);
54 C = make_clouds(r,b,D);
55 if ~C.ok || any(C.tb>D.T(1)), y=1e4; return; end
56 y=-coverage_time(r,b,1,D,.04,full);
57 end
58
59 function [r,b,C,tf,one,tc,gain] = assess(x,D)
60 [r,b] = decode(x);
61 C = make_clouds(r,b,D);

```

```

62 tc = coverage_time(r,b,1,D,.005,false);
63 tf = coverage_time(r,b,1,D,.01,true);
64 one=zeros(3,1); gain=zeros(3,1);
65 for i=1:3
66     one(i)=coverage_time(r,b(i,:),1,D,.01,true);
67     keep=true(3,1); keep(i)=false;
68     gain(i)=tf-coverage_time(r,b(keep,:),1,D,.01,true);
69 end
70 end
71
72 function [r,b] = decode(x)
73 r=zeros(5,2); r(1,:)=x(1:2);
74 td=[x(3),x(3)+1+x(4),x(3)+2+x(4)+x(5)]';
75 b=[ones(3,1),td,x(6:8)'];
76 end

```

Listing 14 问题 4 求解 solve_problem4.m

```

1 base_dir = fileparts(mfilename('fullpath'));
2 result_dir = fullfile(base_dir,'..','result');
3 if ~exist(result_dir,'dir'), mkdir(result_dir); end
4
5 D = data_a();
6 mid = 1;
7 lb = repmat([0 70 0 0],1,3);
8 ub = zeros(1,12);
9 x = zeros(1,12);
10 for k = 1:3
11     q = (k-1)*4+(1:4);
12     ub(q) = [360 140 D.T(mid) sqrt(2*D.u0(k,3)/D.g)];
13     fk = @(z) cost_seed(z,k,mid,D);
14     [x(q),~] = de_search(fk,lb(q),ub(q),46,75,400+k);
15 end
16 S2 = load(fullfile(result_dir,'problem2.mat'),'x');
17 x(1:4) = S2.x;
18 f = @(z) cost_multi3(z,mid,D,0.05,false);
19 [x,~] = local_search(f,x,lb,ub,repmat([10 8 2 1],1,3),5);
20 ff = @(z) cost_multi3(z,mid,D,0.05,true);
21 [x,~] = local_search(ff,x,lb,ub,repmat([8 6 1 0.7],1,3),3);
22 [routes,bombs] = decode_multi3(x);
23 C = make_clouds(routes,bombs,D);
24 t_center = coverage_time(routes,bombs,mid,D,0.005,false);
25 t_full = coverage_time(routes,bombs,mid,D,0.01,true);
26 single_time = zeros(3,1);
27 for i = 1:3
28     single_time(i) = coverage_time(routes,bombs(i,:),mid,D,0.01,true);
29 end

```

```

30 save(fullfile(result_dir,'problem4.mat'),'x','routes','bombs','C', ...
31     't_center','t_full','single_time');
32 fprintf('P4: %.3f s, full %.3f s\n',t_center,t_full);
33
34 function y = cost_seed(x,uav,mid,D)
35 routes = zeros(5,2);
36 routes(uav,:) = x(1:2);
37 bombs = [uav x(3:4)];
38 C = make_clouds(routes,bombs,D);
39 if ~C.ok || C.tb>D.T(mid), y = 1e4; return; end
40 y = -coverage_score(routes,bombs,mid,D,0.1);
41 end
42
43 function y = cost_multi3(x,mid,D,dt,full_target)
44 [routes,bombs] = decode_multi3(x);
45 C = make_clouds(routes,bombs,D);
46 if ~C.ok || any(C.tb > D.T(mid)), y = 1e4; return; end
47 y = -coverage_time(routes,bombs,mid,D,dt,full_target);
48 end
49
50 function [routes,bombs] = decode_multi3(x)
51 routes = zeros(5,2);
52 bombs = zeros(3,3);
53 for k = 1:3
54     q = (k-1)*4+(1:4);
55     routes(k,:) = x(q(1:2));
56     bombs(k,:) = [k x(q(3:4))];
57 end
58 end

```

Listing 15 问题 5 求解 solve_problem5.m

```

1 base_dir = fileparts(mfilename('fullpath'));
2 result_dir = fullfile(base_dir,'..','result');
3 if ~exist(result_dir,'dir'), mkdir(result_dir); end
4 D = data_a();
5
6 cand = repmat(struct('x',[],'route',[],'bombs',[]),5,3);
7 for k = 1:5
8     for j = 1:3
9         tm = sqrt(2*D.u0(k,3)/D.g);
10        lb = [0 70 0 0 0 0 0 0];
11        ub = [360 140 32 13 13 tm tm tm];
12        f = @(x) cost_uav(x,k,j,D,0.1);
13        [x,~] = de_search(f,lb,ub,30,42,500+10*k+j);
14        [x,~] = local_search(f,x,lb,ub,[10 8 1 1 1 0.8 0.8 0.8],2);
15        [r,b] = decode_uav(x,k);

```

```

16         cand(k,j).x = x;
17         cand(k,j).route = r(k,:);
18         cand(k,j).bombs = b;
19     end
20 end
21
22 combo_assign = zeros(243,5);
23 combo_score = zeros(243,3);
24 for code = 0:242
25     a = mod(floor(code./(3.^(0:4))),3)+1;
26     [routes,bombs] = join_plan(cand,a);
27     combo_assign(code+1,:) = a;
28     combo_score(code+1,:) = coverage_time(routes,bombs,1:3,D,0.1,true)';
29 end
30
31 total = sum(combo_score,2);
32 best_sum = max(total);
33 near = find(total >= best_sum-0.05);
34 worst = min(combo_score(near,:),[],2);
35 top = near(worst == max(worst));
36 [~,id] = max(total(top));
37 assign = combo_assign(top(id),:);
38 candidate_assign = assign;
39
40 xs = cell(5,1);
41 for k = 1:5, xs{k} = cand(k,assign(k)).x; end
42 [r0,b0] = join_xs(xs);
43 xs0 = xs;
44 sum_floor = sum(coverage_time(r0,b0,1:3,D,0.08,true))-0.05;
45 for k = 1:5
46     tm = sqrt(2*D.u0(k,3)/D.g);
47     lb = [0 70 0 0 0 0 0 0];
48     ub = [360 140 32 13 13 tm tm tm];
49     f = @(z) cost_context(z,k,xs,D,sum_floor);
50     [xs{k},~] = local_search(f,xs{k},lb,ub,[6 5 0.7 0.7 0.7 0.5 0.5 0.5],2)
51     ;
52 end
53 [routes,bombs] = join_xs(xs);
54 f0 = coverage_time(r0,b0,1:3,D,0.02,true);
55 f1 = coverage_time(routes,bombs,1:3,D,0.02,true);
56 if sum(f0)>sum(f1)+0.05 || (sum(f0)>=sum(f1)-0.05 && min(f0)>min(f1))
57     xs=xs0; routes=r0; bombs=b0;
58 end
59 [bombs,removed,kept] = prune_bombs(routes,bombs,D,.02);
60 [routes,bombs] = refine_plan(routes,bombs,D);
61 [bombs,removed2,kept] = prune_bombs(routes,bombs,D,.02,kept);

```

```

61 removed = [removed; removed2];
62 added_seed=zeros(0,3);
63 bomb_id=kept;
64 next_id=16;
65 for cycle=1:9
66     [r_keep,b_keep] = deal(routes,bombs);
67     keep_score = coverage_time(routes,bombs,1:3,D,.02,true);
68     [b_try,a_try] = add_bombs(routes,bombs,D);
69     [r_try,b_try] = refine_plan(routes,b_try,D);
70     new_score = coverage_time(r_try,b_try,1:3,D,.02,true);
71     gain_sum=sum(new_score)-sum(keep_score);
72     gain_min=min(new_score)-min(keep_score);
73     if gain_sum>.01 || (gain_sum>=-.02 && gain_min>.01)
74         routes=r_try; bombs=b_try;
75         added_seed=[added_seed;a_try]; %#ok<AGROW>
76         nadd=size(a_try,1);
77         bomb_id=[bomb_id;(next_id:next_id+nadd-1)']; %#ok<AGROW>
78         next_id=next_id+nadd;
79     else
80         routes=r_keep; bombs=b_keep; break
81     end
82 end
83 [bombs,removed_final,bomb_id] = prune_bombs(routes,bombs,D,.02,bomb_id);
84 [bombs,ord] = sortrows(bombs,[1 2]);
85 bomb_id=bomb_id(ord);
86 C = make_clouds(routes,bombs,D);
87 t_center = coverage_time(routes,bombs,1:3,D,0.01,false);
88 t_full = coverage_time(routes,bombs,1:3,D,0.02,true);
89
90 primary = zeros(size(bombs,1),1);
91 marginal_time = zeros(size(bombs,1),1);
92 independent_time = zeros(size(bombs,1),1);
93 for i = 1:size(bombs,1)
94     keep = true(size(bombs,1),1); keep(i)=false;
95     d = max(0,t_full-coverage_time(routes,bombs(keep,:),1:3,D,.02,true));
96     [marginal_time(i),primary(i)] = max(d);
97     independent_time(i) = coverage_time(routes,bombs(i,:),primary(i),D,.02,
98         true);
99 end
100 save(fullfile(result_dir,'problem5.mat'),'cand','combo_assign','combo_score',
101     '...',
102     'candidate_assign','routes','bombs','C','t_center','t_full','primary',
103     '...',
104     'marginal_time','independent_time','bomb_id','removed','removed_final',
105     'added_seed');

```

```

103 fprintf('P5: %.3f s, full %.3f s\n',sum(t_center),sum(t_full));
104
105 function [b,removed,original_id] = prune_bombs(r,b,D,tol,original_id)
106 base = coverage_time(r,b,1:3,D,.02,true);
107 floor_sum = sum(base)-tol;
108 floor_min = min(base)-tol;
109 removed = zeros(0,1);
110 if nargin<5, original_id = (1:size(b,1))'; end
111 while size(b,1)>1
112     best=-inf; pick=0;
113     for i=1:size(b,1)
114         keep=true(size(b,1),1); keep(i)=false;
115         s=coverage_time(r,b(keep,:),1:3,D,.02,true);
116         if sum(s)>=floor_sum && min(s)>=floor_min && sum(s)>best
117             best=sum(s); pick=i;
118         end
119     end
120     if pick==0, break; end
121     removed(end+1,1)=original_id(pick); %#ok<AGROW>
122     original_id(pick)=[];
123     b(pick,:)=[];
124 end
125 end
126
127 function [r,b] = refine_plan(r,b,D)
128 base = coverage_time(r,b,1:3,D,.02,true);
129 for pass = 1:2
130     for k = unique(b(:,1))'
131         id=find(b(:,1)==k); n=numel(id);
132         z=[r(k,:),reshape(b(id,2:3)',1,[])];
133         tm=sqrt(2*D.u0(k,3)/D.g);
134         lb=[0 70,repmat([0 0],1,n)];
135         ub=[360 140,repmat([max(D.T) tm],1,n)];
136         f=@(x) block_cost(x,k,id,r,b,D);
137         [z,~]=local_search(f,z,lb,ub,[4 4,repmat([1 .5],1,n)],4);
138         [rt,bt]=put_block(z,k,id,r,b);
139         s=coverage_time(rt,bt,1:3,D,.02,true);
140         if sum(s)>sum(base)+.01 || (sum(s)>=sum(base)-.02 && min(s)>min(
            base)+.01)
141             r=rt; b=bt; base=s;
142         end
143     end
144 end
145 end
146
147 function y = block_cost(z,k,id,r,b,D)

```

```

148 [r,b]=put_block(z,k,id,r,b);
149 C=make_clouds(r,b,D);
150 td=sort(b(id,2));
151 if ~C.ok || any(C.tb>max(D.T)) || (numel(td)>1 && min(diff(td))<1)
152     y=1e4; return
153 end
154 s=coverage_time(r,b,1:3,D,.05,true);
155 y=-sum(s)-1e-3*min(s);
156 end
157
158 function [r,b] = put_block(z,k,id,r,b)
159 r(k,:)=z(1:2);
160 b(id,2:3)=reshape(z(3:end),2,[])';
161 end
162
163 function y = cost_uav(x,k,j,D,dt)
164 [routes,bombs] = decode_uav(x,k);
165 C = make_clouds(routes,bombs,D);
166 if ~C.ok || any(C.tb>D.T(j)), y = 1e4; return; end
167 y = -coverage_score(routes,bombs,j,D,dt);
168 end
169
170 function y = cost_context(z,k,xs,D,sum_floor)
171 tmp = xs;
172 tmp{k} = z;
173 [routes,bombs] = join_xs(tmp);
174 C = make_clouds(routes,bombs,D);
175 if ~C.ok || any(C.tb>max(D.T)), y = 1e4; return; end
176 s = coverage_time(routes,bombs,1:3,D,0.08,true);
177 if sum(s) < sum_floor
178     y = 100 + 100*(sum_floor-sum(s));
179 else
180     y = -min(s) - 1e-3*sum(s);
181 end
182 end
183
184 function [routes,bombs] = decode_uav(x,k)
185 routes = zeros(5,2);
186 routes(k,:) = x(1:2);
187 td = [x(3), x(3)+1+x(4), x(3)+2+x(4)+x(5)]';
188 bombs = [k*ones(3,1), td, x(6:8)]';
189 end
190
191 function [routes,bombs] = join_plan(cand,a)
192 routes = zeros(5,2);
193 bombs = zeros(0,3);

```

```

194 for k = 1:5
195     routes(k,:) = cand(k,a(k)).route;
196     bombs = [bombs; cand(k,a(k)).bombs]; %#ok<AGROW>
197 end
198 end
199
200 function [routes,bombs] = join_xs(xs)
201 routes = zeros(5,2);
202 bombs = zeros(0,3);
203 for k = 1:5
204     [r,b] = decode_uav(xs{k},k);
205     routes(k,:) = r(k,:);
206     bombs = [bombs; b]; %#ok<AGROW>
207 end
208 end

```

Listing 16 问题 5 整数规划认证 solve_problem5_milp.m

```

1 function solve_problem5_milp(dt)
2 if nargin<1, dt=.2; end
3 base=fileparts(mfilename('fullpath'));
4 out=fullfile(base,'..','result');
5 S=load(fullfile(out,'problem5.mat'));
6 D=data_a(12,3);
7
8 plans=struct('uav',{},'route',{},'bombs',{},'name',{});
9 for k=1:5
10     r=zeros(5,2); r(k,:)=S.routes(k,:);
11     b=S.bombs(S.bombs(:,1)==k,:);
12     plans(end+1)=pack(k,r,b,'final'); %#ok<AGROW>
13     for j=1:3
14         r=zeros(5,2); r(k,:)=S.cand(k,j).route;
15         plans(end+1)=pack(k,r,S.cand(k,j).bombs,sprintf('M%d',j)); %#ok<
            AGROW>
16     end
17 end
18 np=numel(plans);
19
20 cov=cell(np,3); times=cell(3,1);
21 for p=1:np
22     for j=1:3
23         [times{j},cov{p,j}]=plan_point_cover(plans(p).route,plans(p).bombs,
            j,D,dt);
24     end
25 end
26
27 ny=sum(cellfun(@numel,times));

```



```

28 nvar=np+ny;
29 f=zeros(nvar,1);
30 offset=np;
31 A=[]; bineq=[];
32 for j=1:3
33     nt=numel(times{j}); nq=size(cov{1,j},2);
34     C=false(nt*nq,np);
35     for p=1:np, C(:,p)=reshape(cov{p,j}',[],1); end
36     left=-sparse(C);
37     q=(offset-np)+(1:nt);
38     rr=(1:nt*nq)'; cc=repelem(q',nq);
39     right=sparse(rr,cc,1,nt*nq,ny);
40     A=[A;left,right]; %#ok<AGROW>
41     bineq=[bineq;zeros(nt*nq,1)]; %#ok<AGROW>
42     w=dt*ones(nt,1); w([1 end])=dt/2;
43     f(offset+(1:nt))=-w;
44     offset=offset+nt;
45 end
46
47 G=sparse([plans.uav],1:np,1,5,nvar);
48 A=[A;G]; bineq=[bineq;ones(5,1)];
49 opts=optimoptions('intlinprog','Display','final','RelativeGapTolerance',0,'
    MaxTime',300);
50 [z,val,flag,output]=intlinprog(f,1:nvar,A,bineq,[],[],zeros(nvar,1),ones(
    nvar,1),opts);
51 selected=find(z(1:np)>.5);
52
53 routes=zeros(5,2); bombs=zeros(0,3);
54 for p=selected'
55     k=plans(p).uav;
56     routes(k,:)=plans(p).route(k,:);
57     bombs=[bombs;plans(p).bombs]; %#ok<AGROW>
58 end
59 bombs=sortrows(bombs,[1 2]);
60 t_full=coverage_time(routes,bombs,1:3,data_a(48,9),.01,true);
61 milp_time=-val;
62 save(fullfile(out,'problem5_milp.mat'),'plans','selected','routes','bombs',
    ...
    'milp_time','t_full','flag','output','dt');
63 fprintf('MILP %.3f s, strict %.3f s, flag %d\n',milp_time,sum(t_full),flag)
64 ;
65 disp([selected,[plans(selected).uav]]);
66 end
67
68 function p = pack(k,r,b,name)
69 p=struct('uav',k,'route',r,'bombs',b,'name',name);

```

70 end

Listing 17 结果约束验证 validate_results.m

```
1 function report = validate_results()
2 base = fileparts(mfilename('fullpath'));
3 out = fullfile(base, '..', 'result');
4 report = struct([]);
5 Dfine = data_a(48,9);
6
7 for p = 1:5
8     S = load(fullfile(out, sprintf('problem%d.mat', p)));
9     v = S.routes(:,2); v = v(v>0);
10    gaps = inf;
11    for k = unique(S.bombs(:,1))'
12        td = sort(S.bombs(S.bombs(:,1)==k,2));
13        if numel(td)>1, gaps = min(gaps, min(diff(td))); end
14    end
15    if isinf(gaps), gaps = nan; end
16    mids = 1;
17    if p==5, mids=1:3; end
18    fine = coverage_time(S.routes, S.bombs, mids, Dfine, 0.01, true);
19    marginal = zeros(size(S.bombs,1), numel(mids));
20    for i = 1:size(S.bombs,1)
21        keep = true(size(S.bombs,1),1); keep(i)=false;
22        marginal(i,:) = fine' - coverage_time(S.routes, S.bombs(keep,:), mids,
23            Dfine, 0.01, true)';
24    end
25    report(p).speed = [min(v), max(v)];
26    report(p).min_height = min(S.C.burst(:,3));
27    report(p).min_gap = gaps;
28    report(p).fine_time = fine;
29    report(p).marginal = marginal;
30    report(p).valid = S.C.ok && all(v>=70 & v<=140) && ...
31        (isnan(gaps) || gaps>=1-1e-8);
32
33 save(fullfile(out, 'validation.mat'), 'report');
34 for p = 1:5
35     fprintf('P%d valid=%d fine=', p, report(p).valid);
36     fprintf('%.3f ', report(p).fine_time);
37     fprintf('marginal>0: %d/%d\n', sum(any(report(p).marginal>1e-6,2)), size(
38         report(p).marginal,1));
39 end
end
```

Listing 18 离散收敛检验 convergence_check.m

```

1 function conv = convergence_check()
2 base = fileparts(mfilename('fullpath'));
3 out = fullfile(base, '..', 'result');
4 cfg = [24 5; 48 9; 72 13];
5 dt = [0.04 0.02 0.01];
6 conv = cell(5,1);
7 for p = 1:5
8     S = load(fullfile(out, sprintf('problem%d.mat', p)));
9     mids = 1; if p==5, mids=1:3; end
10    a = zeros(3,3,numel(mids));
11    for i = 1:3
12        D = data_a(cfg(i,1),cfg(i,2));
13        for j = 1:3
14            a(i,j,:) = coverage_time(S.routes,S.bombs,mids,D,dt(j),true);
15        end
16    end
17    conv{p} = a;
18    fprintf('P%d coarse %.3f fine %.3f\n',p,sum(a(1,1,:)),sum(a(3,3,:)));
19 end
20 save(fullfile(out,'convergence.mat'),'conv','cfg','dt');
21 end

```

Listing 19 执行误差分析 sensitivity_check.m

```

1 function stat = sensitivity_check(n)
2 if nargin<1, n=100; end
3 base = fileparts(mfilename('fullpath'));
4 out = fullfile(base, '..', 'result');
5 S = load(fullfile(out, 'problem5.mat'));
6 D = data_a(24,5);
7 rng(825);
8
9 sample = zeros(n,3);
10 for h = 1:n
11     r = S.routes;
12     b = S.bombs;
13     for k = 1:5
14         r(k,1) = mod(r(k,1) + .4*rand-.2,360);
15         r(k,2) = min(140,max(70,r(k,2)*(1+.02*rand-.01)));
16         id = find(b(:,1)==k);
17         shift = .1*rand-.05;
18         shift = max(shift,-min(b(id,2)));
19         b(id,2) = b(id,2)+shift;
20     end
21     b(:,3) = max(0,b(:,3)+.1*rand(size(b,1),1)-.05);
22     sample(h,:) = coverage_time(r,b,1:3,D,.04,true)';

```

```

23 end
24
25 total = sum(sample,2);
26 stat.base = coverage_time(S.routes,S.bombs,1:3,D,.04,true)';
27 stat.mean = mean(sample,1);
28 stat.std = std(sample,0,1);
29 stat.min = min(sample,[],1);
30 stat.max = max(sample,[],1);
31 stat.total = [sum(stat.base),mean(total),std(total),min(total),max(total)];
32 stat.n = n;
33 save(fullfile(out,'sensitivity.mat'),'stat','sample');
34 fprintf('Sensitivity total: base %.3f, mean %.3f, std %.3f, min %.3f\n',
        stat.total(1:4));
35 end

```

Listing 20 结果表导出 export_results.m

```

1 function export_results()
2 base_dir = fileparts(mfilename('fullpath'));
3 root = fullfile(base_dir,'..');
4 src = fullfile(root,'题目','附件');
5 out = fullfile(root,'result');
6 if ~exist(out,'dir'), mkdir(out); end
7
8 S = load(fullfile(out,'problem3.mat'));
9 dst = fullfile(out,'result1.xlsx');
10 copyfile(fullfile(src,'result1.xlsx'),dst,'f');
11 A = [repmat(S.routes(1,:),3,1), (1:3)', S.C.drop, S.C.burst, S.single_time
    ];
12 writematrix(A,dst,'Range','A2');
13
14 S = load(fullfile(out,'problem4.mat'));
15 dst = fullfile(out,'result2.xlsx');
16 copyfile(fullfile(src,'result2.xlsx'),dst,'f');
17 A = cell(3,10);
18 for i = 1:3
19     A(i,:) = [{sprintf('FY%d',i)}, num2cell([S.routes(i,:),S.C.drop(i,:),
        ...
        S.C.burst(i,:),S.single_time(i))]);
20 end
21 writecell(A,dst,'Range','A2');
22
23
24 S = load(fullfile(out,'problem5.mat'));
25 dst = fullfile(out,'result3.xlsx');
26 copyfile(fullfile(src,'result3.xlsx'),dst,'f');
27 A = cell(15,12);
28 for k = 1:5

```

```

29     rows = (k-1)*3+(1:3);
30     A(rows,1) = repmat({sprintf('FY%d',k)},3,1);
31     A(rows,4) = num2cell((1:3)');
32     id = find(S.bombs(:,1)==k);
33     for q = 1:numel(id)
34         i = id(q); row = rows(q);
35         A(row,:) = [{sprintf('FY%d',k)},num2cell(S.routes(k,:)),{q}, ...
36                     num2cell(S.C.drop(i,:)),num2cell(S.C.burst(i,:)), ...
37                     {S.independent_time(i)},{sprintf('M%d',S.primary(i))}];
38     end
39 end
40 writecell(A,dst,'Range','A2');
41 end

```

Listing 21 论文图像生成 make_figures.m

```

1 base=fileparts(mfilename('fullpath'));
2 figdir=fullfile(base,'figures');
3 if ~exist(figdir,'dir'),mkdir(figdir);end
4 addpath(fullfile(base,'..','code'));
5 D=data_a();
6
7 S=load(fullfile(base,'..','result','problem5.mat'));
8 f=figure('Color','w','Position',[100 100 1080 620]); hold on; grid on; box on;
9 col=lines(5);
10 for k=1:5
11     id=find(S.bombs(:,1)==k);
12     if isempty(id),continue;end
13     tt=linspace(0,max(S.C.tb(id)),100)';
14     e=[cosd(S.routes(k,1)),sind(S.routes(k,1)),0];
15     xyz=D.u0(k,:)+S.routes(k,2)*tt.*e;
16     plot3(xyz(:,1),xyz(:,2),xyz(:,3),'LineWidth',1.8,'Color',col(k,:), ...
17           'DisplayName',sprintf('FY%d 航迹',k));
18     scatter3(S.C.burst(id,1),S.C.burst(id,2),S.C.burst(id,3),48,col(k,:), ...
19             'filled','HandleVisibility','off');
20 end
21 mc=[.15 .15 .15;.45 .45 .45;.70 .25 .15];
22 for j=1:3
23     tt=linspace(0,D.T(j),120)'; e=-D.m0(j,:)/norm(D.m0(j,:));
24     xyz=D.m0(j,:)+D.v0*tt.*e;
25     plot3(xyz(:,1),xyz(:,2),xyz(:,3),'--','Color',mc(j,:), ...
26           'LineWidth',1.3,'DisplayName',sprintf('M%d 轨迹',j));
27 end
28 scatter3(nan,nan,nan,48,'k','filled','DisplayName','烟幕起爆点');
29 scatter3(0,200,5,90,'r','p','filled','DisplayName','真目标');
30 xlabel('x/m');ylabel('y/m');zlabel('z/m');
31 view(38,24); axis tight;
32 legend('Location','eastoutside','NumColumns',1);
33 exportgraphics(f,fullfile(figdir,'trajectory_p5.png'),'Resolution',300); close(f);
34
35 I=coverage_intervals(S.routes,S.bombs,1:3,D,.01);
36 f=figure('Color','w','Position',[100 100 900 360]); hold on; box on;
37 cc=lines(3);
38 for j=1:3
39     a=I(j).covered;

```

```

40     for r=1:size(a,1)
41         rectangle('Position',[a(r,1),4-j,a(r,2)-a(r,1),.55], ...
42             'FaceColor',cc(j,:), 'EdgeColor','none');
43     end
44 end
45 yticks([1.275 2.275 3.275]); yticklabels({'M3','M2','M1'});
46 xlabel('时间/s');
47 xlim([0 40]); ylim([.8 4]); grid on;
48 exportgraphics(f,fullfile(figdir,'intervals_p5.png'),'Resolution',300); close(f);
49
50 S4=load(fullfile(base,'..','result','problem4.mat'));
51 f=figure('Color','w','Position',[100 100 720 430]);
52 v=[S4.single_time;S4.t_full];
53 b=bar(v,'FaceColor','flat');
54 b.CData(1:3,:)=repmat([.20 .50 .75],3,1);
55 b.CData(4,:)=[.90 .40 .10];
56 set(gca,'XTickLabel',{'FY1单弹','FY2单弹','FY3单弹','联合方案'});
57 ylabel('有效遮蔽时间/s'); grid on; ylim([0 12.5]);
58 text(1:4,v+.22,compose('% .3f',v),'HorizontalAlignment','center');
59 exportgraphics(f,fullfile(figdir,'problem4_bar.png'),'Resolution',300); close(f);
60
61 S3=load(fullfile(base,'..','result','problem3.mat'));
62 I3=coverage_intervals(S3.routes,S3.bombs,1,D,.01);
63 f=figure('Color','w','Position',[100 100 820 360]); hold on; box on;
64 cc=lines(4);
65 for i=1:3
66     Ii=coverage_intervals(S3.routes,S3.bombs(i,:),1,D,.01);
67     a=Ii.covered;
68     for r=1:size(a,1)
69         rectangle('Position',[a(r,1),4.05-i,a(r,2)-a(r,1),.38], ...
70             'FaceColor',cc(i,:), 'EdgeColor','none');
71     end
72 end
73 a=I3.covered;
74 for r=1:size(a,1)
75     rectangle('Position',[a(r,1),.75,a(r,2)-a(r,1),.38], ...
76         'FaceColor',cc(4,:), 'EdgeColor','none');
77 end
78 ylim([.6 3.7]); yticks([.94 1.24 2.24 3.24]);
79 yticklabels({'联合','弹3','弹2','弹1'}); grid on;
80 xlabel('时间/s'); xlim([0 8]);
81 text(.25,1.24,'0 s (无独立遮蔽)','Color',[.35 .35 .35], ...
82     'VerticalAlignment','middle');
83 exportgraphics(f,fullfile(figdir,'intervals_p3.png'),'Resolution',300); close(f);
84
85 R=load(fullfile(base,'..','result','sensitivity.mat'));
86 f=figure('Color','w','Position',[100 100 760 420]);
87 histogram(sum(R.sample,2),12,'FaceColor',[.20 .50 .75],'EdgeColor','w');
88 xline(R.stat.total(1),'r--','基准值','LineWidth',1.4, ...
89     'LabelOrientation','horizontal','LabelVerticalAlignment','middle', ...
90     'LabelHorizontalAlignment','left');
91 xline(R.stat.total(2),'k-','扰动均值','LineWidth',1.4, ...
92     'LabelOrientation','horizontal','LabelVerticalAlignment','top', ...
93     'LabelHorizontalAlignment','left');
94 xlabel('三枚导弹总遮蔽时间/s'); ylabel('频数');
95 grid on;
96 yl=ylim; text(min(sum(R.sample,2))+.3,.92*yl(2),'n=100','FontWeight','bold');
97 exportgraphics(f,fullfile(figdir,'sensitivity_hist.png'),'Resolution',300); close(f);

```

```

98
99 C=load(fullfile(base,'.','result','convergence.mat'));
100 coarse=zeros(5,1);fine=zeros(5,1);
101 for p=1:5
102     coarse(p)=sum(C.conv{p}(1,1,:));
103     fine(p)=sum(C.conv{p}(3,3,:));
104 end
105 f=figure('Color','w','Position',[100 100 760 430]);
106 err=abs(coarse-fine);
107 bar(1:5,err,.62,'FaceColor',[.20 .50 .75]);grid on;
108 xticks(1:5);xticklabels({'问题1','问题2','问题3','问题4','问题5'});
109 ylabel('粗、精网格绝对差/s');ylim([0 .07]);
110 text(1:5,err+.002,compose('%.3f',err),'HorizontalAlignment','center');
111 exportgraphics(f,fullfile(figdir,'convergence.png'),'Resolution',300); close(f);

```

附录 C 支撑文件说明

表 12 支撑材料文件列表

文件	功能
code/run_all.m	顺序执行求解、验证、敏感性分析、MILP 认证和制图
code/data_a.m	题目数据与圆柱目标采样
code/make_clouds.m	投放点、起爆点、有效期及约束检查
code/coverage_time.m	完整目标严格遮蔽时长
code/coverage_score.m	连续接近度代理目标
code/coverage_intervals.m	遮蔽区间端点提取
code/de_search.m	差分进化搜索
code/local_search.m	坐标局部修正
code/add_bombs.m	空缺导向的新增烟幕弹搜索
code/plan_point_cover.m	MILP 候选点级遮蔽矩阵
code/solve_problem1.m	问题 1 求解
code/solve_problem2.m	问题 2 求解
code/solve_problem3.m	问题 3 求解
code/solve_problem4.m	问题 4 求解
code/solve_problem5.m	问题 5 分层优化
code/solve_problem5_milp.m	候选集 0-1 整数规划认证
code/validate_results.m	约束、高密度时长和边际贡献验证
code/convergence_check.m	时间与空间离散收敛检验
code/sensitivity_check.m	执行误差随机模拟
code/export_results.m	三个 Excel 结果表导出
paper/make_figures.m	论文轨迹、区间、贡献和收敛图生成
result/problem1.mat 至 problem5.mat	五问完整计算结果
result/problem5_milp.mat	整数规划选择及最优性信息
result/validation.mat	高密度约束验证结果
result/convergence.mat	离散精度收敛结果
result/sensitivity.mat	100 次扰动样本及统计量
result/result1.xlsx 至 result3.xlsx	按赛题模板填写的最终策略
paper/figures/*.png	轨迹、区间、贡献及收敛图像
paper/烟幕干扰弹投放策略.tex	论文 LaTeX 源文件